StudyForge

Stage 1 Project Design Proposal

Adaptive Study and Learning Agent

EECS3311 Software Design | Fall 2026
Zaaid Ahmed | York University
2 October 2026

Proposal scope

This report will define the planned architecture, agent behavior, features, UML models, design patterns, use cases, runtime interactions, and implementation traceability for StudyForge. Every design statement will describe intended future behavior.

Report map and proposal basis

Section	Planned content
1	Problem, motivation, model boundary, agent responsibilities, layered architecture
2	Fourteen non-trivial features with type, interface, inputs, outputs, workflow and alternatives
3	Five applied design patterns and their proposed participants
4	Actors and 14 detailed use cases
5	PlantUML sequence flows for grounding, planning, and study-plan generation
6	Complete feature-to-design mapping
7	Execution walkthrough for every feature

Design status: This Stage 1 report specifies intended behavior and software structure. Implementation details may be refined in Stage 2 while preserving traceability from features and use cases to classes and interactions.

Scope: The first design target will be one local student workspace. The system will process PDF, TXT, and Markdown study material, use structured learning data in H2, and access an LLM through a provider boundary.

Authentication, multi-user cloud hosting, OCR, speech, embeddings, and vector search will remain outside the initial proposal scope.

1. Project overview and architecture

1.1 Problem and motivation

University students will often need to search lecture notes, readings, and revision material while also deciding what to practise next. Manual searching, making practice questions, tracking weak areas, and scheduling review across separate documents will consume study time. A general chatbot may answer without evidence from the student's course, while a conventional file organizer will not turn retrieved material into explanations or targeted practice.

StudyForge will target university and college students who study from course PDFs, plain-text notes, and Markdown files. The proposed system will keep each course as a separate workspace and will connect source material to generated explanations and practice results.

A single LLM request will not address the full study workflow: StudyForge will retrieve course evidence, validate tool input and model output, persist learning results, and track progress over time. For a clearly recognized request, IntentAnalyzer and TaskPlanner will select a predetermined plan. For an ambiguous or explicitly multi-step goal, TaskPlanner may ask the local LLM to propose a bounded sequence of typed tool steps. AgentPlanValidator will check the course, intent, tool allowlist, and step count before execution; invalid proposals will follow a safe deterministic fallback. The LLM will also support grounded explanations, summaries, concepts, flashcards, quizzes and source-checked short-answer grading.

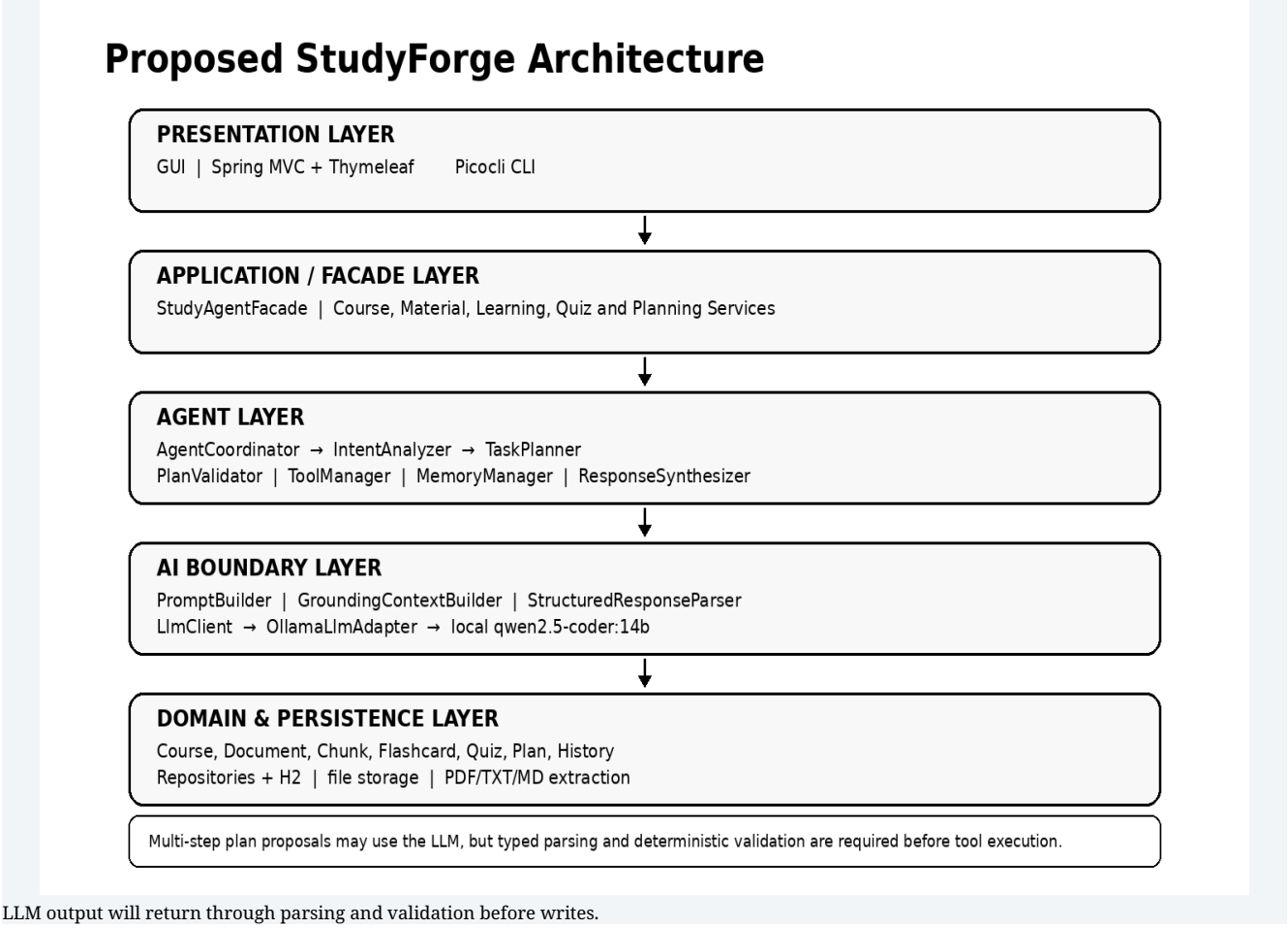
1.2 AI component and interaction boundary

The proposed model target will be qwen2.5-coder:14b served by the local Ollama runtime, using the configured base URL and http://localhost:11434 as the default. LlmClient will define provider-neutral generation and availability operations, while OllamaLlmAdapter will translate application requests into Ollama HTTP requests and map responses or runtime errors to application result types. IntentAnalyzer will handle clear keyword-driven requests deterministically. For ambiguous or explicitly multi-step study goals, TaskPlanner will be able to request a structured plan proposal through LlmClient. AgentPlanValidator will then enforce the supported intent set, course scope, tool allowlist and step limit before any tool executes. If planning output is unavailable or invalid, TaskPlanner will use a deterministic fallback plan.

PromptBuilder will construct task-specific prompts. GroundingContextBuilder will label source excerpts and enforce a bounded context size. StructuredResponseParser will parse structured responses. Feature services will validate required fields, allowed values, source identifiers, size limits, and course ownership before returning or saving generated objects. Model output will be treated as untrusted input: it will not write to persistence or bypass deterministic validation.

The application will remain useful when the model is unavailable for deterministic actions such as course administration, import, lexical search, review history, objective quiz grading, progress calculations, and rule-based scheduling. AI-dependent features will return a clear availability result and preserve any already-saved user data.

1.3 Proposed system architecture



LLM output will return through parsing and validation before writes.

Web controllers and Picocli commands will share StudyAgentFacade and focused application services for supported operations. The presentation layer will keep request parsing at the boundary; application services will validate and coordinate use cases. The agent layer will combine deterministic intent rules with bounded LLM-assisted plan proposals for multi-step requests, validate every proposed tool step, and execute only registered services. The AI boundary will isolate Ollama transport, prompts, and structured response parsing. Domain repositories and document infrastructure will own persisted course and learning data.

The proposed design includes both GUI and Picocli CLI access to major workflows, fourteen non-trivial features, a local Ollama model boundary, retrieval and bounded memory, validated tool execution, and multi-step task handling. Clear requests will use deterministic routing, while ambiguous or explicitly multi-step requests may use the LLM to propose a typed plan that must pass deterministic validation before execution. MVC, Facade, Adapter, Strategy and Command will address five distinct design problems in the system.

Operational scope will remain local-first: PDF, TXT and Markdown course materials; H2-backed course and learning records; local file storage; and the configured Ollama provider boundary. OCR, cloud hosting, embeddings, vector search, authentication, and multi-user administration will remain outside this proposal.

1.4 Requirements, constraints and applicability

Concern	Proposal
Language and runtime	Java 21 with Maven dependency management
GUI	Spring Boot MVC controllers and Thymeleaf views
CLI	Picocli command objects sharing application services
Persistence	Spring Data JPA with H2 for local structured records
Document processing	PDFBox for PDF text extraction; UTF-8 text paths for TXT and Markdown; normalization and chunking
LLM	Ollama local runtime; qwen2.5-coder:14b as initial configurable model
Boundary behavior	Connection and request timeouts, availability checks, structured parsing, bounded retries
Workspace model	Course-scoped files and records; local-first single-user operation

2. Detailed feature specification

The proposed release contains fourteen distinct features. Deterministic components will handle course/material validation, storage, lexical retrieval, objective grading, progress calculations, rule-based recommendations, and study-plan constraints. The configured model will handle bounded content generation and short-answer evaluation. For ambiguous or explicitly multi-step requests, it may also propose a typed agent plan that must pass deterministic validation before a tool can run.

F01 Course workspace management

AI involvement: Deterministic

Description: Students will create, view, update and remove course workspaces in the GUI; the CLI will provide course creation, listing and display. Course records will keep learning material and activities scoped to the selected course.

User interaction:

- GUI: dashboard and course forms support create, list, show, update and delete.
- CLI: course create, list and show.

Input: Course code, title, optional description and exam date.

Output: A validated course record and updated course list.

Expected workflow:

1) Receive and validate course fields.
2) Normalize the course code and check uniqueness.
3) Create, update, retrieve or delete the course through CourseService.
4) Persist and return CourseResponse values.

- A persistence constraint may refuse deletion when dependent records remain.

Error / alternative cases:

- Blank fields, duplicate codes and missing courses will produce validation or not-found outcomes.
- A delete that conflicts with dependent database records may be refused by persistence.
- No cascade or dependency-cleanup behavior will be promised.

F02 Course material import and processing

AI involvement: Deterministic

Description: Students will add PDF, TXT, or Markdown files. The importer will validate, store, extract, normalize, and split content into source-addressable chunks.

User interaction:

- GUI: course materials upload.
- CLI: material add/list/show.

Input: Course identifier and file bytes or path; supported extension and configured size ceiling.

Output: CourseDocument, ordered DocumentChunk records, import status and source metadata.

Expected workflow:

1) Confirm course.
2) Validate type, size, content and duplicate hash.
3) Store under generated safe name.
4) Select extractor.
5) Normalize and chunk text.
6) Persist metadata and chunks transactionally.

Error / alternative cases:

- Unsupported, oversized, empty, corrupt, duplicate, or textless inputs will be rejected or marked failed.
- Partial file writes will trigger cleanup.
- OCR will be out of scope.

F03 Course material retrieval

AI involvement: Deterministic

Description: Students and agent tools will find relevant passages using explainable lexical matching and course-scoped ranking.

User interaction:

- GUI: material search.
- CLI: material search.
- Agent: COURSE_SEARCH tool.

Input: Course, query, optional document restriction and bounded top-K.

Output: Ranked SearchResult values with document, chunk, page and score metadata.

Expected workflow:

1) Validate query and scope.
2) Normalize query.
3) Load READY chunks for the selected course.
4) Score term and phrase overlap.
5) Apply deterministic tie-breaking and top-K.
6) Return source-linked matches.

Error / alternative cases:

- Blank query will fail validation.
- Zero results will return guidance.
- Repository issues will return a service error.
- Retrieval in this version is lexical.
- Semantic or vector retrieval is outside scope.

F04 Grounded course question answering

AI involvement: Hybrid

Description: Students will ask questions and receive explanations grounded in retrieved excerpts from a course or selected document.

User interaction:

- GUI: Ask StudyForge and Ask this document.
- CLI: ask.
- The Q&A workflow will use the direct grounded-question service; it will not be exposed as an AgentTool.

Input: Course, natural-language question and optional document scope.

Output: GroundedAnswer with an answer, validated source IDs and grounded/insufficient-context status.

Expected workflow:

1) Validate the question and course/document scope.
2) Retrieve course-scoped chunks.
3) Return an insufficiency result without a model call when retrieval is empty.
4) Build bounded evidence and a prompt.
5) Call the configured model and parse its structured response.
6) Check that cited source IDs belong to retrieved context.
7) Return the answer with provenance.

Error / alternative cases:

- No retrieved evidence will return an insufficiency result without model generation.
- Model unavailability, invalid structured output, blank output or unknown source IDs will return a controlled error.
- Unsupported claims will not be presented as grounded.

F05 Document and topic summarization

AI involvement: Hybrid

Description: Students will condense a selected document or course topic into an overview and source-linked review points.

User interaction:

- GUI: summarize a document or topic.
- CLI: summary document/topic.
- Agent tool: GENERATE_SUMMARY.

Input: Course and a READY document or a topic phrase for summary scope.

Output: SummaryResult containing overview, key points and source references.

Expected workflow:

1) Resolve the selected READY document or retrieve chunks for a topic phrase.
2) Bound and label evidence.
3) Build a summary prompt.
4) Request structured output.
5) Validate required fields and cited source IDs.
6) Return SummaryResult to the interface without persisting it as a learning record.

Error / alternative cases:

- Missing or unready source, no relevant topic chunks, provider failure, malformed output, or invalid citations will produce a clear error or insufficiency result.
- The service will use the configured context bound.
- Hierarchical summarization is outside scope.

F06 Key concept extraction

AI involvement: Hybrid

Description: Students will identify important terms and receive concise explanations tied to their selected study material.

User interaction:

- GUI: concepts view/action.
- CLI: concepts extract.
- Concept extraction will be available as a direct feature service, not as an AgentTool.

Input: Course and selected document. Concept extraction will operate on a document, not a free-text topic query.

Output: Deduplicated ExtractedConcept list with term, definition and source identifiers.

Expected workflow:

1) Validate scope.
2) Retrieve bounded chunks.
3) Prompt model for structured concept candidates.
4) Parse candidates.
5) Normalize names and definitions.
6) Merge duplicates and verify source IDs.
7) Return concepts.

Error / alternative cases:

- No usable context will yield an empty/insufficient result.
- Incomplete fields, invalid citations or provider failure will return a controlled failure.
- Duplicate concepts will be merged deterministically.

F07 Adaptive flashcard generation and review

AI involvement: Hybrid

Description: Students will generate source-grounded flashcards and record review ratings. Ratings will contribute to progress and later recommendations.

User interaction:

- GUI: generate, list, reveal and rate.
- CLI: flashcards generate/list.
- Agent tool: GENERATE_FLASHCARDS; review ratings will remain a direct user workflow.

Input: Course, ready document, requested count; for review, card ID and rating.

Output: Persisted Flashcard and provenance records; append-only FlashcardReview events.

Expected workflow:

1) Validate course, READY document and count.
2) Build bounded source context.
3) Request structured question/answer/source triples.
4) Validate fields and source IDs, remove duplicates and persist valid cards.
5) Reveal answers during review.
6) Validate the selected rating and append a FlashcardReview record.
7) ProgressService will use the saved review when its summary is requested.

Error / alternative cases:

- Missing or unready source, model unavailability, invalid generated fields, or invalid source IDs will return a controlled result.
- Valid cards may be saved with the created count reported.
- Missing/foreign cards or invalid ratings will fail safely.
- Saved cards will remain reviewable if the LLM is unavailable.

F08 Grounded quiz generation

AI involvement: Hybrid

Description: Students will generate reusable quizzes from selected course material using supported question formats.

User interaction:

- GUI: select a ready document, question count, and supported question types.
- CLI: quiz generate/list.
- Agent tool: GENERATE_QUIZ using the first ready document.

Input: Course, one ready document, requested count and permitted question types. The interface will not request a difficulty setting.

Output: Quiz, QuizQuestion and QuizSource records with requested and retained counts.

Expected workflow:

1) Validate count, types and source scope.
2) Retrieve evidence.
3) Request structured questions.
4) Parse and filter duplicates and invalid answer structures.

5) Verify source references.

6) Persist the valid quiz and report partial completion.

Error / alternative cases:

- If the first structured response cannot be processed, one bounded retry will use smaller context.
- If no question passes validation, no quiz will be saved.
- Partial completion will be reported.

F09 Quiz taking and grading

AI involvement: Hybrid

Description: Students will submit answers and receive per-question feedback and a saved score. Objective grading will be deterministic; short answers will use source-checked LLM evaluation when available and a disclosed deterministic fallback otherwise.

User interaction:

- GUI: quiz generation, taking, submission and results.
- CLI: quiz generate/list only; taking and submission will remain GUI flows.

Input: Course-owned quiz and question-to-answer map.

Output: QuizAttempt, QuizResponse values, score and grading explanation.

Expected workflow:

1) Load the course-owned quiz.

2) Create an attempt and grade objective items deterministically.

3) For short answers, use source-checked model evaluation when available and a deterministic word-overlap fallback when unavailable or invalid.

4) Store responses and completion time.

5) Return the score and per-question feedback; later progress and history queries will read the saved attempt.

Error / alternative cases:

- Blank answers will be recorded as unanswered/incorrect.
- A foreign quiz or overlong answer will be rejected or marked incorrect according to the service behavior.
- If short-answer grading is unavailable or cannot be validated, deterministic word-overlap grading will be used and its fallback message will be returned.

F10 Evidence-based wrong-answer explanation

AI involvement: Hybrid

Description: Students will request a teaching explanation for a response, with course evidence and a reference answer where available.

User interaction:

- GUI: explain from quiz result.
- This feature will not be exposed as a CLI command or agent tool.

Input: Course, quiz and question identifiers, and the student's submitted answer.

Output: GroundedExplanation with a teaching explanation, correction and validated source citations.

Expected workflow:

1) Confirm quiz and question ownership.
2) Load question references and cited chunks.
3) Build prompt with answer, expected response and evidence.
4) Generate structured teaching feedback.
5) Validate output and citations.
6) Display without changing recorded grade.

Error / alternative cases:

- If no source can be recovered, explanation will state the limitation.
- Provider errors and invalid responses will not alter the stored attempt or grade.

F11 Progress and weak-area analysis

AI involvement: Deterministic

Description: Students will see transparent progress signals calculated from quiz attempts and flashcard reviews, with low-scoring groups flagged for review.

User interaction:

- GUI: progress dashboard.
- CLI: progress.
- Agent: GET_PROGRESS and GET_WEAK_TOPICS tools.

Input: Course and relevant saved review/attempt records.

Output: ProgressSummary, per-topic/group score, activity count and status classification.

Expected workflow:

1) Validate course.
2) Load saved flashcard reviews and quiz attempts.
3) Map ratings and quiz outcomes to fixed numeric scores.
4) Group flashcard reviews by source document and quiz attempts under quiz performance.
5) Compute averages and fixed status bands.
6) Return overall and per-group progress.

Error / alternative cases:

- No activity will return a no-data state.
- The calculation will use fixed score bands.
- Minimum sample-size thresholds and configurable classification policies are outside this version.

F12 Personalized study-plan generation

AI involvement: Deterministic

Description: Students will receive a dated plan that distributes one total available study-time budget across the chosen date range. The selected strategy will determine whether activities are balanced, weak-topic focused, or exam-cram style; no per-day limit or exam-date proximity calculation will be specified.

User interaction:

- GUI: study-plan form/list.

- CLI: study-plan create/list.
- Agent tool: CREATE_STUDY_PLAN with its configured default dates, total minutes and strategy.

Input: Course, start date, target date, total available minutes and one of the supported study strategies.

Output: Persisted StudyPlan with dated StudyPlanItem activities and the submitted total-minute budget.

Expected workflow:

1) Validate the date range, positive total minutes and strategy.
2) Load course progress.
3) Ask the selected StudyPlanningStrategy to distribute the total minutes across dates and topic labels.
4) Reject a generated allocation that exceeds the submitted total.
5) Persist the plan and its dated activities.
6) Display the saved plan.

Error / alternative cases:

- A missing or reversed date range, nonpositive minutes, unknown strategy, or over-budget generated schedule will return a validation error.
- With no progress topics, the balanced strategy will schedule course-material review.
- No daily cap or adjustment recommendation will be claimed.

F13 Adaptive next-step recommendations and agent task planning

AI involvement: Hybrid

Description: Students will request a recommendation or bounded study workflow. RecommendationService will use deterministic progress/history rules. AgentCoordinator will use keyword intent rules for clear requests and bounded LLM-assisted plan proposals for ambiguous or explicitly multi-step goals. Every proposed plan will be converted to typed steps, validated against the allowlist, and executed with bounded learning history; selected tools may also call the LLM for grounded content generation.

User interaction:

- GUI: Ask StudyForge, recommendation page.
- CLI: agent assist and recommendation commands.

Input: Course and natural-language goal. Recent learning events will be retrieved automatically after tool execution for response synthesis.

Output: A deterministic recommendation, or response text listing the selected intent, tool results/statuses, and the count of recent learning events considered.

Expected workflow:

1) IntentAnalyzer will map clear wording to a supported intent.
2) For an ambiguous or explicitly multi-step request, TaskPlanner will build a bounded planning prompt and ask LlmClient for a structured plan proposal; otherwise it will use the deterministic plan for the recognized intent.
3) StructuredResponseParser will parse the proposal into typed steps.
4) AgentPlanValidator will enforce the course requirement, supported intent, tool allowlist and one-to-five step limit.
5) AgentToolManager will execute the validated services.
6) After execution, MemoryManager will retrieve ten recent course events.

7) ResponseSynthesizer will format the selected intent, tool outcomes and memory count.

- Selected content-generation tools will continue to use their own grounded model workflows.

Error / alternative cases:

- If a request is not confidently matched by the deterministic rules, TaskPlanner may request a structured model proposal.
- Invalid, malformed, over-length or unsupported plans will be rejected and replaced by a safe deterministic progress-review plan.
- Unsupported tools and service failures will be returned as controlled tool results.
- No unvalidated model-selected tool will execute.

F14 Learning history and bounded agent memory

AI involvement: Deterministic

Description: Students will inspect chronological flashcard-review, quiz-attempt and study-plan activity. Agent memory will provide the ten most recent course events to response synthesis.

User interaction:

- GUI: history page.
- CLI: history.
- Agent: GET_HISTORY and MemoryManager.

Input: Course; history list will be capped by the service and agent memory will request the ten most recent events.

Output: Chronological HistoryEvent list of up to 100 course events; bounded agent memory of ten recent HistoryEvent values.

Expected workflow:

1) Query flashcard-review, quiz-attempt and study-plan records for the course.
2) Map them to HistoryEvent values.
3) Sort newest first and cap the GUI/CLI list at 100.
4) After agent tool execution, MemoryManager will request the ten newest events and pass them to response synthesis.

- No topic filter will be provided.

Error / alternative cases:

- No events will produce an empty state.
- The history records will not contain Q&A transcripts, agent plans, material-import events, or hidden model reasoning.

3. Proposed design patterns

The following five patterns will address distinct responsibilities in the proposed architecture. Their participants will remain subject to Stage 2 refinement, but the design will preserve the roles and contracts listed here.

Model–View–Controller (MVC)

Participants: CourseController, MaterialController, AskController, QuizController, Thymeleaf views, domain/application services.

Participant roles: Controllers will translate HTTP input into facade calls, services and entities will form the model, and Thymeleaf templates will render views.

Design problem and rationale: MVC will keep HTTP form handling and page rendering separate from business rules. Without this separation, GUI concerns would spread into reusable services and make CLI reuse harder.

Facade

Participants: StudyAgentFacade; web controllers and Picocli commands as clients; CourseService, learning services and AgentCoordinator as subsystem participants.

Participant roles: The facade will expose stable use-case methods such as importMaterial(), askQuestion(), generateQuiz(), submitQuiz(), and assist().

Design problem and rationale: A facade will reduce client knowledge of internal collaborator graphs and preserve a shared application entry point for GUI and CLI. It will prevent each boundary from coordinating repositories and AI components independently.

Adapter

Participants: LlmClient interface; OllamaLlmAdapter; Ollama HTTP API; PromptBuilder and feature services as clients.

Participant roles: LlmClient will define provider-neutral generation and availability methods. OllamaLlmAdapter will translate requests, timeouts, and responses.

Design problem and rationale: The adapter will prevent Ollama request formats and transport errors from coupling feature logic to one vendor. Tests will substitute a fake LlmClient without starting a model runtime.

Strategy

Participants: StudyPlanningStrategy; BalancedStudyStrategy, WeakTopicFocusedStrategy, ExamCramStrategy; StudyPlanService.

Participant roles: Each strategy will implement a shared plan-building contract. StudyPlanService will choose a policy, supply progress/context, enforce global budget constraints, and persist the result.

Design problem and rationale: Strategies will allow planning policies to vary without a long conditional scheduler. A common output contract will make each policy independently testable and replaceable.

Command

Participants: Picocli CommandLine invoker; CourseCreateCommand, CourseListCommand, CourseShowCommand, MaterialSearchCommand, QuizCommand.Generate, QuizCommand.ListQuizzes, and AgentCommand; StudyAgentFacade as the application receiver.

Participant roles: Picocli CommandLine will act as the invoker. Each dedicated command object (such as CourseCreateCommand or MaterialSearchCommand) will encapsulate one operation and its command-line arguments through run() or call(). StudyAgentFacade will serve as the receiver and delegate to the appropriate application service. AgentToolManager will remain a separate execution dispatcher; it will not be counted as the Command pattern.

Design problem and rationale: Command objects will keep parsing and validation of command-line arguments separate from application behavior. Individual commands can be tested and extended without changing a single monolithic CLI switch statement. This design qualifies as Command because the invoker executes distinct objects representing user-requested operations, and those objects delegate to a separate receiver.

4. Use-case specifications

4.1 Actors

Actor	Role in proposed system
Student	Primary user who will manage courses, material, practice, progress, plans, and agent requests.
Local LLM Runtime	Supporting external system that will generate structured language output when an AI feature is selected.
Document Store / File System	Supporting external resource that will hold uploaded source files under generated safe paths.
Timer / System Clock	Supporting system actor that will provide timestamps for attempts, review events, plans, and history.

The GUI will provide the main course, material, learning, quiz-taking and history workflows. The CLI will provide supported course create/list/show, material, AI status, Q&A, summary, concept, flashcard, quiz generate/list, progress, study-plan, history, recommendation and agent commands. Quiz taking/submission/explanation and course update/delete will remain GUI-only. Both interfaces will use the shared facade and application services where their current entry points do so.

UC01 Manage a course

Primary actor(s): Student

Goal: Create or maintain a course workspace.

Preconditions: Application will be running; the selected course will need to exist for show, update or delete. The CLI will expose create, list and show; update and delete will be GUI actions.

Trigger: Student will submit course details or a course command.

Main success scenario:

1. CourseController or a course command will validate fields.
2. The supported operation will call StudyAgentFacade and CourseService.
3. CourseService will normalize the code and check uniqueness.
4. CourseRepository will persist or retrieve the course.
5. The GUI or CLI will display the result.

Alternative / exception flows:

- Invalid or duplicate data will return field errors.
- Missing courses will return a not-found result.
- A delete that conflicts with database foreign-key constraints will be refused by persistence.
- No cascade behavior will be claimed.

Postconditions: Course metadata will reflect the accepted operation.

Related feature ID(s): F01

UC02 Import course material

Primary actor(s): Student; Local File System as a supporting actor for reading/storing imported material.

Goal: Make a supported file available as searchable course evidence.

Preconditions: Target course will exist; file will be accessible.

Trigger: Student will select a file or path and start import.

Main success scenario:

1. Boundary will pass bytes and metadata.
2. Service will validate type/size/hash.
3. Storage will save safe copy.
4. Extractor will read text.
5. Normalizer/chunker will create source-addressable chunks.
6. Repositories will persist them.

Alternative / exception flows:

- Unsupported, oversized, duplicate, empty, or corrupt input will fail with reason.
- Partial storage will be cleaned up.

Postconditions: Successful import will create READY document and chunks.

Related feature ID(s): F02

UC03 Search course material

Primary actor(s): Student

Goal: Find passages relevant to a query.

Preconditions: Course will exist; searchable chunks may be empty.

Trigger: Student will submit a nonblank query.

Main success scenario:

1. UI/CLI will validate and send scope.
2. Search service will load course chunks.
3. Scorer will rank lexical/phrase matches.
4. Bounded results will include provenance.

Alternative / exception flows:

- No hits will return an empty-state suggestion.
- Invalid query will be rejected.

Postconditions: No records will change; ranked results will be displayed.

Related feature ID(s): F03

UC04 Ask a grounded course question

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Receive a source-backed explanation.

Preconditions: Course will exist. A grounded answer will require relevant retrieved evidence and an available model.

Trigger: Student will submit a question.

Main success scenario:

1. AskController or AskCommand will call StudyAgentFacade.askQuestion().
2. GroundedQuestionService will retrieve course-scoped chunks.
3. Empty retrieval will return insufficiency without an LLM call.
4. GroundingContextBuilder and PromptBuilder will form bounded evidence and an LlmRequest.
5. LlmClient and OllamaLlmAdapter will request a structured answer.
6. StructuredResponseParser and source validation will check the output.
7. The interface will show the answer or controlled status.

Alternative / exception flows:

- No evidence will produce insufficiency without a model call.
- Provider, parsing or citation failures will produce a controlled status without an automatic retry.

Postconditions: A grounded answer or insufficiency result will be returned.

Related feature ID(s): F04

UC05 Summarize a document or topic

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Obtain a concise review of selected course evidence.

Preconditions: Course and usable source scope will exist.

Trigger: Student will request document or topic summary.

Main success scenario:

1. Service will load/retrieve chunks.
2. Context will be bounded.
3. Model will return structured summary.
4. Parser and citation validator will verify output.
5. Result will be presented.

Alternative / exception flows:

- No relevant chunks or unavailable model will produce a recoverable error.
- Long contexts will be bounded.

Postconditions: Summary will be returned with evidence references.

Related feature ID(s): F05

UC06 Extract key concepts

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Identify terms and explanations in a selected source.

Preconditions: Course/document will belong together and contain extractable text.

Trigger: Student will invoke concept extraction.

Main success scenario:

1. Service will collect context.

2. Model will propose structured concepts.
3. Parser will validate fields.
4. Normalizer will merge duplicates.
5. Source IDs will be checked.

Alternative / exception flows:

- Malformed concepts or unsupported references will be dropped or reported.
- Empty context will return no concepts.

Postconditions: Validated concept list will be returned.

Related feature ID(s): F06

UC07 Generate and review flashcards

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Create source-grounded flashcards and record a review.

Preconditions: Generation will require ready source and model; review will require owned card.

Trigger: Student will request generation or submit a rating.

Main success scenario:

1. FlashcardGenerationService will retrieve one ready document and request structured cards.
2. It will validate source IDs and persist valid cards.
3. The student will reveal a card and submit a rating.
4. FlashcardReviewService will append the rating record.
5. ProgressService will read it during a later progress request.

Alternative / exception flows:

- Partial valid set will report count.
- Invalid card/rating or unavailable model will not corrupt prior cards/events.

Postconditions: Cards and/or review events will be persisted.

Related feature ID(s): F07

UC08 Generate a quiz

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Create a reusable evidence-grounded quiz.

Preconditions: Course and ready source will be available.

Trigger: Student will select a ready course document, question count and supported types in the GUI or CLI.

Main success scenario:

1. QuizGenerationService will validate the course document, count and question types.
2. It will assemble bounded document evidence.
3. LlmClient will request structured questions.
4. The parser and deterministic checks will validate question fields and citations.
5. QuizRepository will persist the valid quiz and questions and report the retained count.

Alternative / exception flows:

- A single bounded retry may use a smaller context after structural failure.
- If no valid question remains, no quiz will be saved.
- A partial valid count will be disclosed.

Postconditions: Validated quiz will be available for completion.

Related feature ID(s): F08

UC09 Take and submit a quiz

Primary actor(s): Student; Local LLM Runtime for short-answer evaluation when available.

Goal: Record answers and receive a score.

Preconditions: Quiz will belong to the selected course.

Trigger: Student will submit answers.

Main success scenario:

1. QuizController will receive submitted answers.
2. QuizAttemptService will load the course-owned quiz and create an attempt.
3. Objective answers will be graded deterministically.
4. Short answers will use source-checked LLM grading where possible and a deterministic fallback when unavailable or invalid.
5. QuizResponse records and the completed attempt will be saved.
6. The score and per-question feedback will display.

Alternative / exception flows:

- Blank answers will be marked unanswered/incorrect.
- Invalid quiz ownership or oversized answers will be handled by QuizAttemptService.
- Short-answer model failure or invalid output will use the deterministic overlap fallback with its explanatory message.

Postconditions: Attempt and response records will be persisted; ProgressService and LearningHistoryService will read them when their views are requested.

Related feature ID(s): F09

UC10 Request an answer explanation

Primary actor(s): Student; Local LLM Runtime for this AI-assisted operation.

Goal: Understand a response using course evidence.

Preconditions: Quiz and question will belong to the selected course.

Trigger: Student will choose Explain from the quiz result page.

Main success scenario:

1. QuizController will pass the course, quiz, question and student response to StudyAgentFacade.
2. WrongAnswerExplanationService will check quiz ownership and locate question source chunks.
3. It will build bounded evidence and a grounded prompt.
4. LlmClient will request structured feedback.
5. StructuredResponseParser and source validation will check the response.

6. The GUI will display the explanation without changing the stored grade.

Alternative / exception flows:

- Unavailable model will return an explanation-unavailable result.
- Invalid citations or malformed output will return a safe limitation.
- The saved quiz grade will remain unchanged.

Postconditions: Explanation will be returned without mutating the attempt score.

Related feature ID(s): F10

UC11 Review progress and weak areas

Primary actor(s): Student

Goal: Inspect calculated study signals.

Preconditions: Course will exist; no practice activity may yet exist.

Trigger: Student will open progress or ask the agent.

Main success scenario:

1. ProgressService will load flashcard reviews and quiz attempts.
2. It will map each record to a fixed numeric score.
3. It will group flashcard reviews by source document and quiz outcomes as quiz performance.
4. It will calculate averages and apply fixed status bands.
5. The progress page or agent tool will receive ProgressSummary.

Alternative / exception flows:

- No activity will display the no-data state.
- Repository errors will return a controlled failure.
- Fixed score bands will apply without a sample-size threshold.

Postconditions: A transparent calculated summary will be displayed; source events will remain unchanged.

Related feature ID(s): F11

UC12 Create a study plan

Primary actor(s): Student

Goal: Allocate study time across a date range and topics.

Preconditions: Course will exist; requested dates and budget will be supplied.

Trigger: Student will submit plan constraints.

Main success scenario:

1. StudyPlanController or StudyPlanCommand will pass course, dates, total minutes and strategy to StudyPlanService.create().
2. The service will obtain ProgressSummary topics.
3. The selected strategy will create dated activities.
4. The service will validate the date range and ensure the activity sum does not exceed the submitted total.
5. StudyPlanRepository will save the plan and items.
6. The interface will display the returned plan.

Alternative / exception flows:

- Missing or reversed dates, nonpositive minutes, an unsupported strategy or an over-budget allocation will return a validation error.
- If progress contains no topics, the balanced strategy will distribute review of course material.

Postconditions: A feasible plan will be stored with total allocation within user budget.

Related feature ID(s): F12

UC13 Ask agent for a recommendation or multi-step task

Primary actor(s): Student; Local LLM Runtime when a structured planning proposal or a content-generation tool is requested.

Goal: Get an actionable next step or execute a bounded study workflow.

Preconditions: Application will be running; selected course will be valid.

Trigger: Student will enter natural-language goal.

Main success scenario:

1. AgentCoordinator will pass the request to IntentAnalyzer.
2. A clear request will use the corresponding deterministic intent; an ambiguous or explicitly multi-step request will be sent to TaskPlanner for a bounded structured plan proposal through LlmClient.
3. StructuredResponseParser will convert valid planning output into typed steps.
4. AgentPlanValidator will enforce the supported intent set, course scope, tool allowlist and step limit.
5. AgentToolManager will execute only the validated steps.
6. MemoryManager will load ten recent course events.
7. ResponseSynthesizer will return the selected intent, tool outcomes and memory count.

- AI-backed generation tools will continue to use the configured model through their own services.

Alternative / exception flows:

- If deterministic intent matching is inconclusive, the planner may request a structured model proposal.
- Malformed, unsupported, over-length or otherwise invalid proposals will fall back to the safe progress-review plan.
- Unknown tools will never execute, and tool failures will appear as controlled results.

Postconditions: A recommendation or formatted agent response containing validated tool outcomes will be returned; any model-proposed plan will have passed deterministic validation before execution.

Related feature ID(s): F13

UC14 Inspect learning history and memory

Primary actor(s): Student

Goal: Review saved practice and planning activity and provide bounded recent events to agent response synthesis.

Preconditions: Course will exist.

Trigger: Student will open history or agent will request memory.

Main success scenario:

1. LearningHistoryService will query flashcard reviews, quiz attempts and study plans.

2. Records will map to HistoryEvent and sort newest first.
3. The history page will display up to 100 entries; MemoryManager will request the ten most recent for agent use.

- No topic filtering will be described.

Alternative / exception flows:

- No events will return an empty state.
- The history model will not include hidden model reasoning.

Postconditions: History view/context will be returned without modifying past events.

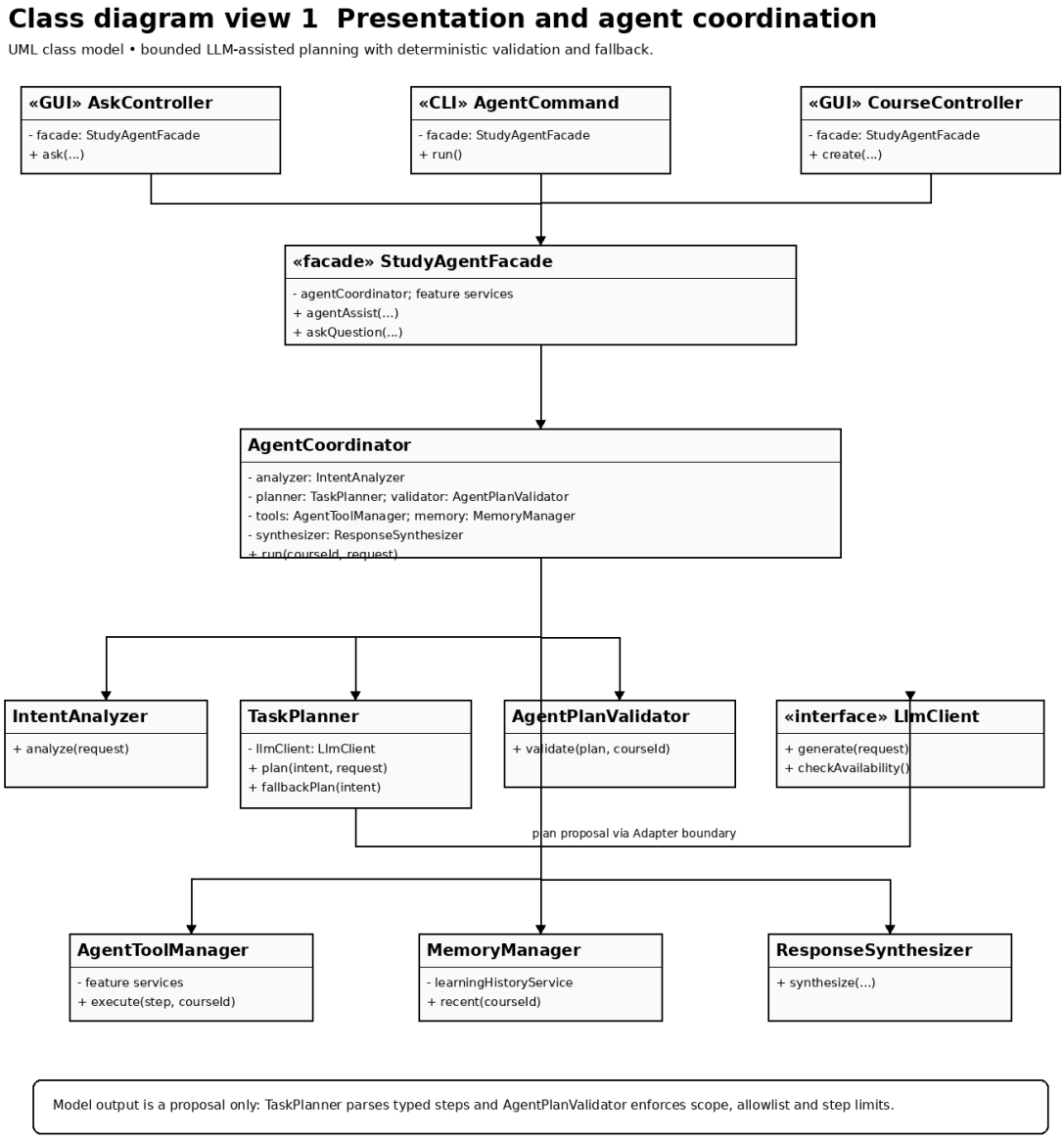
Related feature ID(s): F14

5. UML structural and use-case diagrams

The class diagram is organized into five related UML views to keep the main classes, interfaces, attributes, methods and relationships legible. Together they describe presentation and agent coordination; content processing and AI integration; course-material and flashcard records; quiz persistence; and study planning. The views are proposed design models, not evidence that implementation is complete.

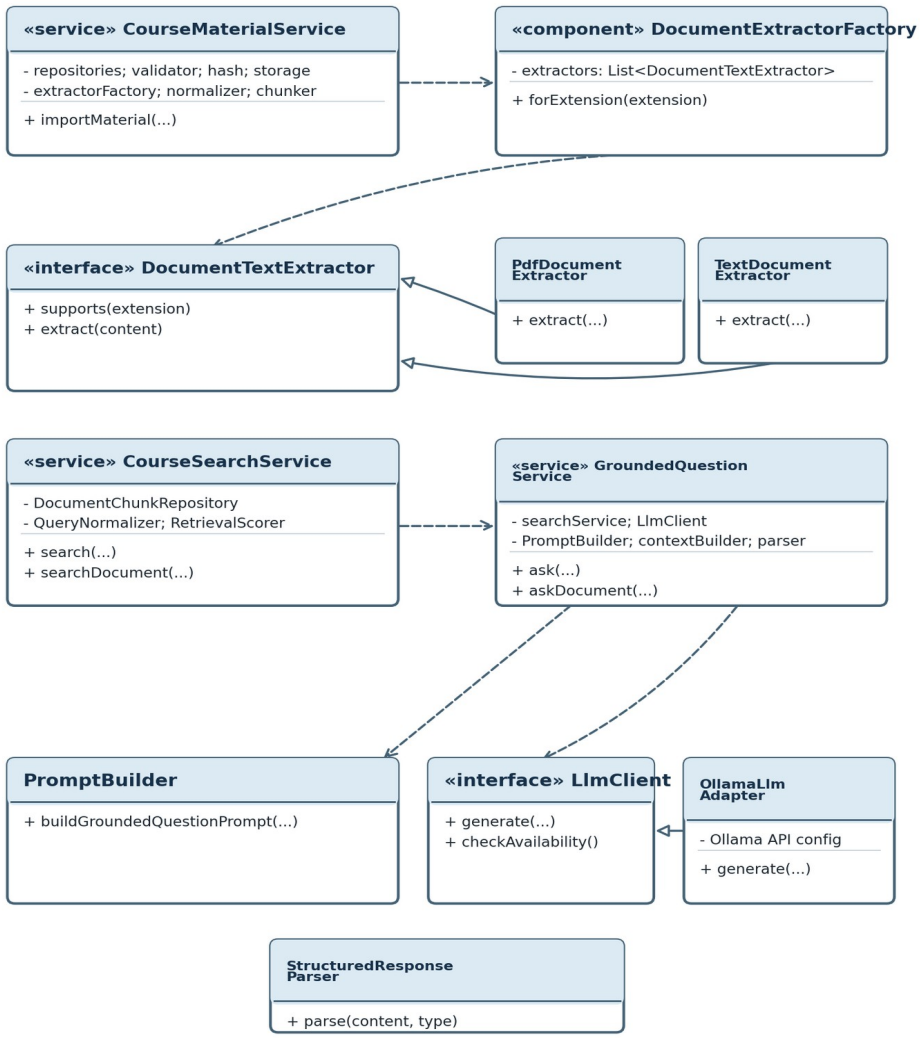
5.1 Proposed class diagram

Class diagram view 1 Presentation and agent coordination



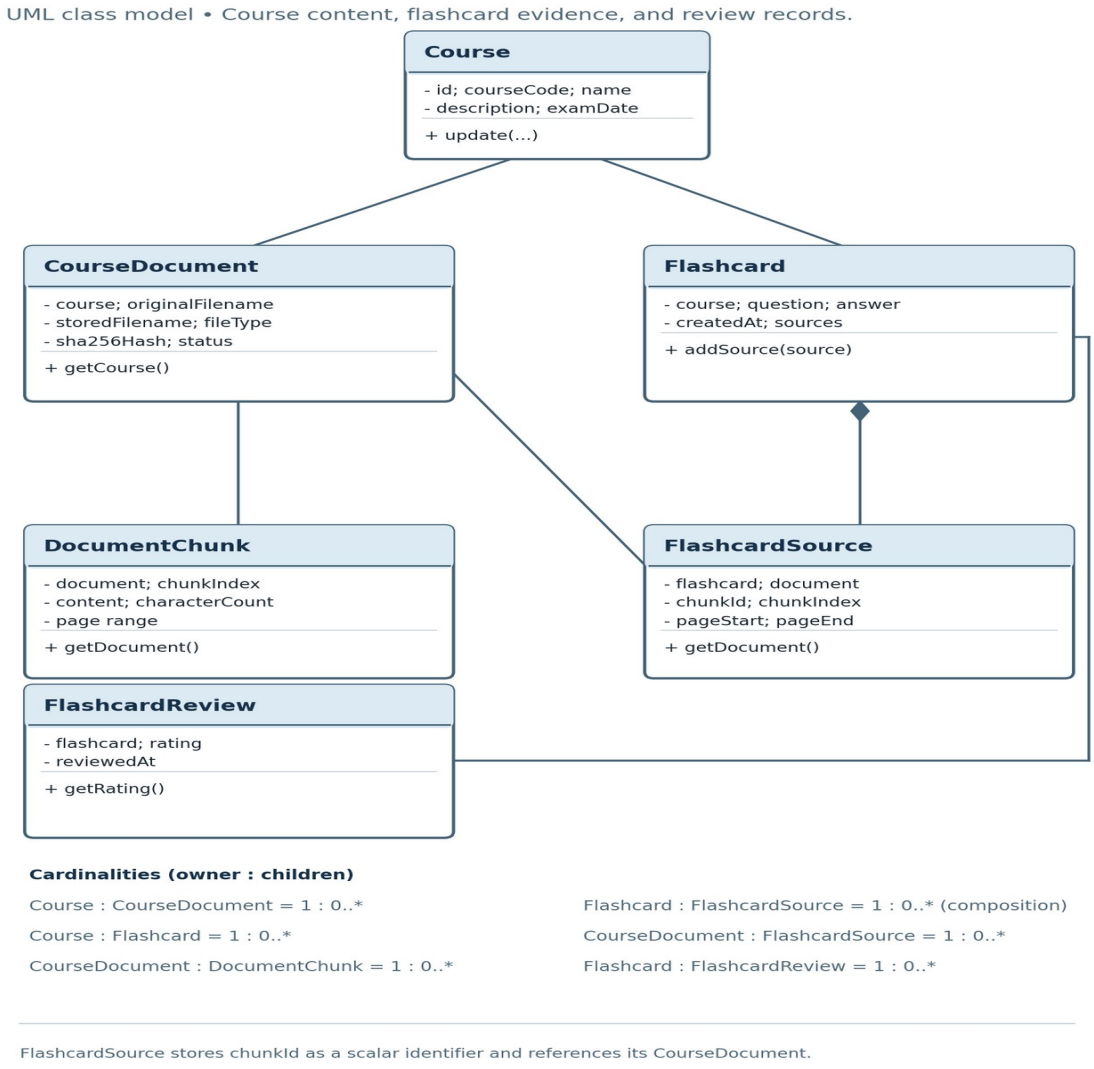
Class diagram view 2 Content pipeline and AI boundary

UML class model • Content import, retrieval, and the model boundary use explicit services and interfaces.

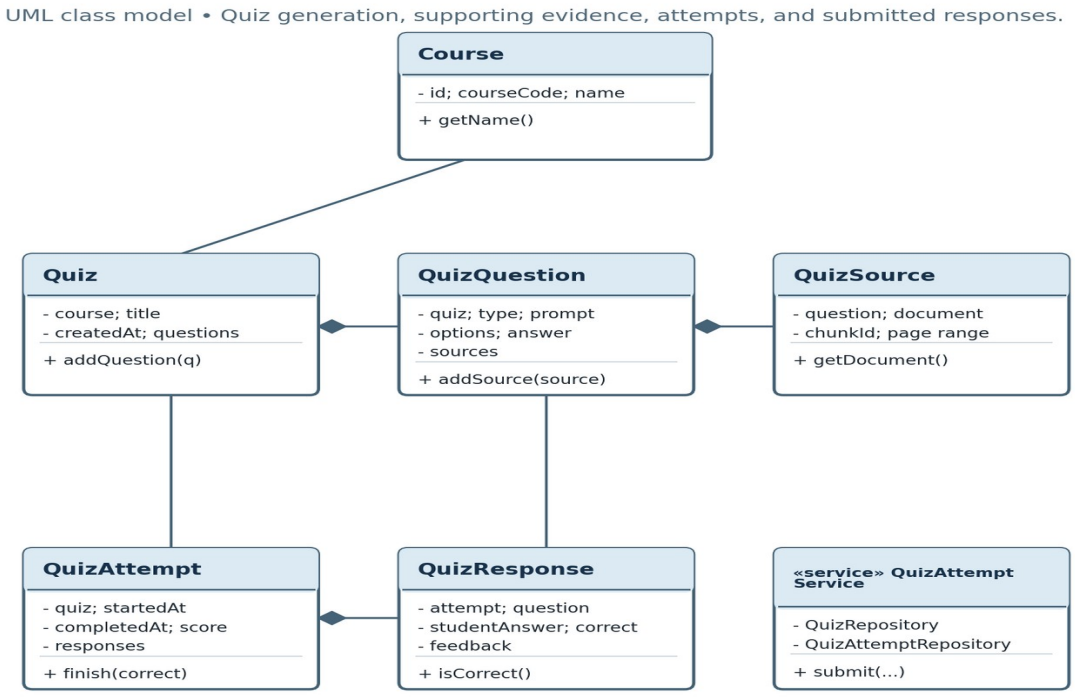


Open triangle = interface implementation. Search uses lexical ranking; provider output is parsed and source IDs are checked by the feature service.

Class diagram view 3 Course materials and flashcard records



Class diagram view 4 Quiz workflow and persistence



Cardinalities

Course–Quiz = 1 : 0..*

Quiz–QuizQuestion = 1 : 1..* (composition)

QuizQuestion–QuizSource = 1 : 0..* (composition)

Quiz–QuizAttempt = 1 : 0..*

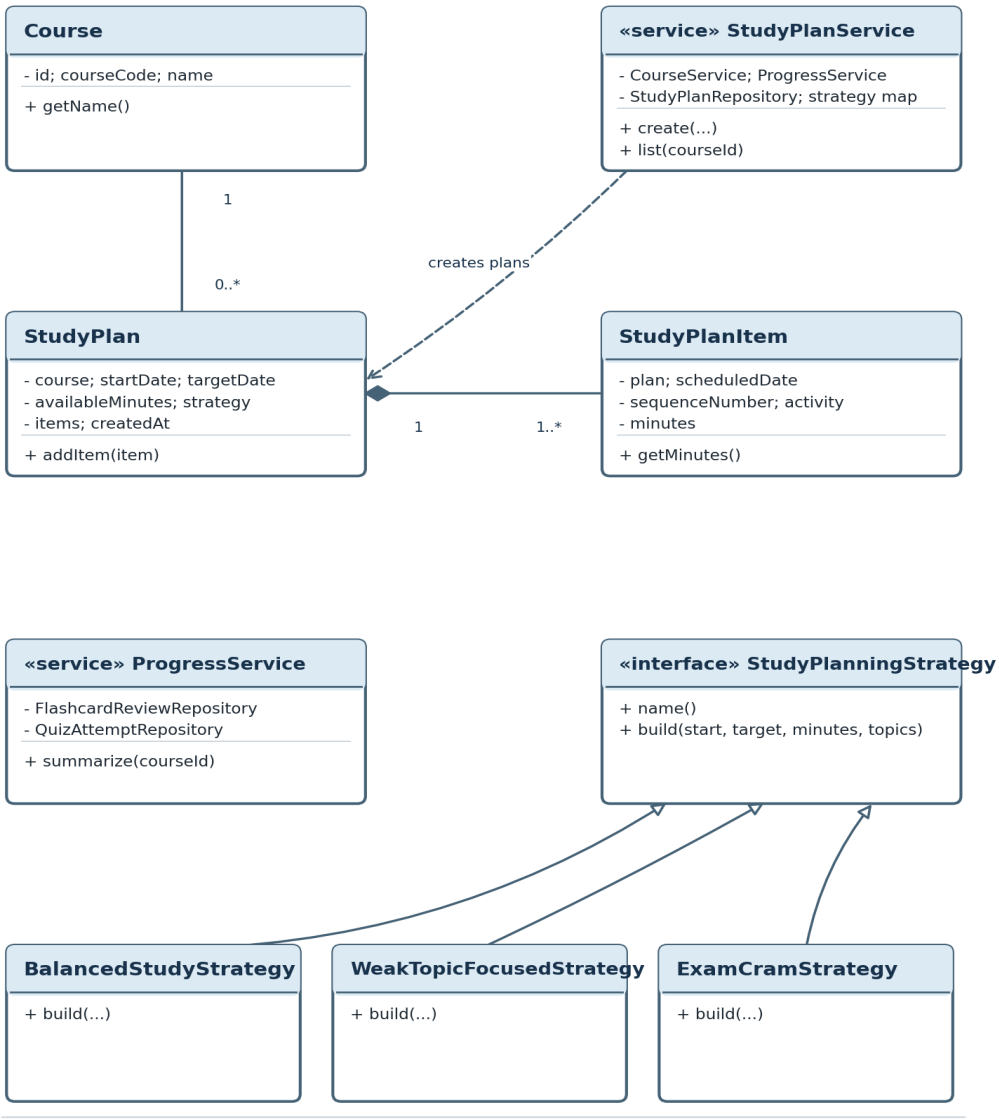
QuizAttempt–QuizResponse = 1 : 0..* (composition)

QuizQuestion–QuizResponse = 1 : 0..*

Each QuizResponse references one QuizQuestion. Source records preserve document and chunk identifiers.

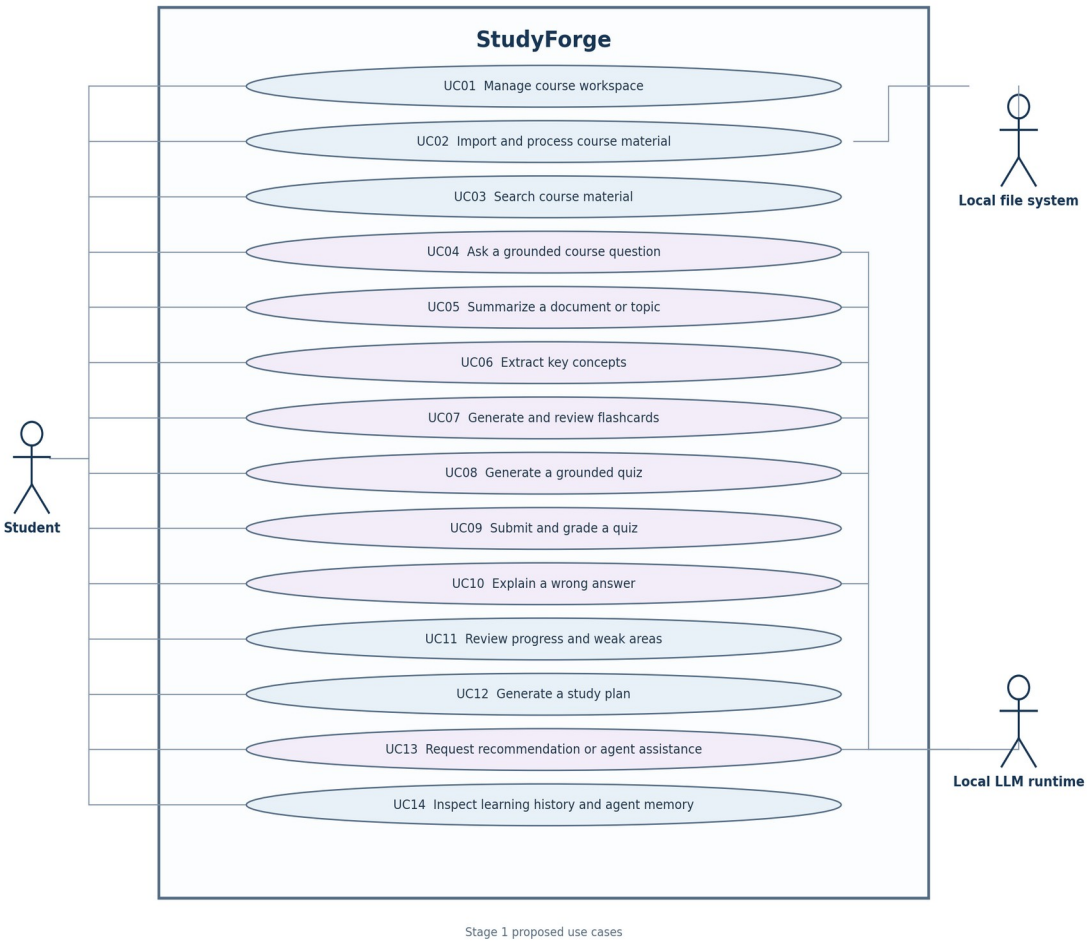
Class diagram view 5 Study plans, progress, and Strategy

UML class model • Progress feeds study plans; concrete scheduling policies implement a Strategy interface.



StudyPlanService selects one strategy. EXAM_CRAM orders weak topics; it does not calculate proximity to Course.examDate.

5.2 Proposed use-case diagram

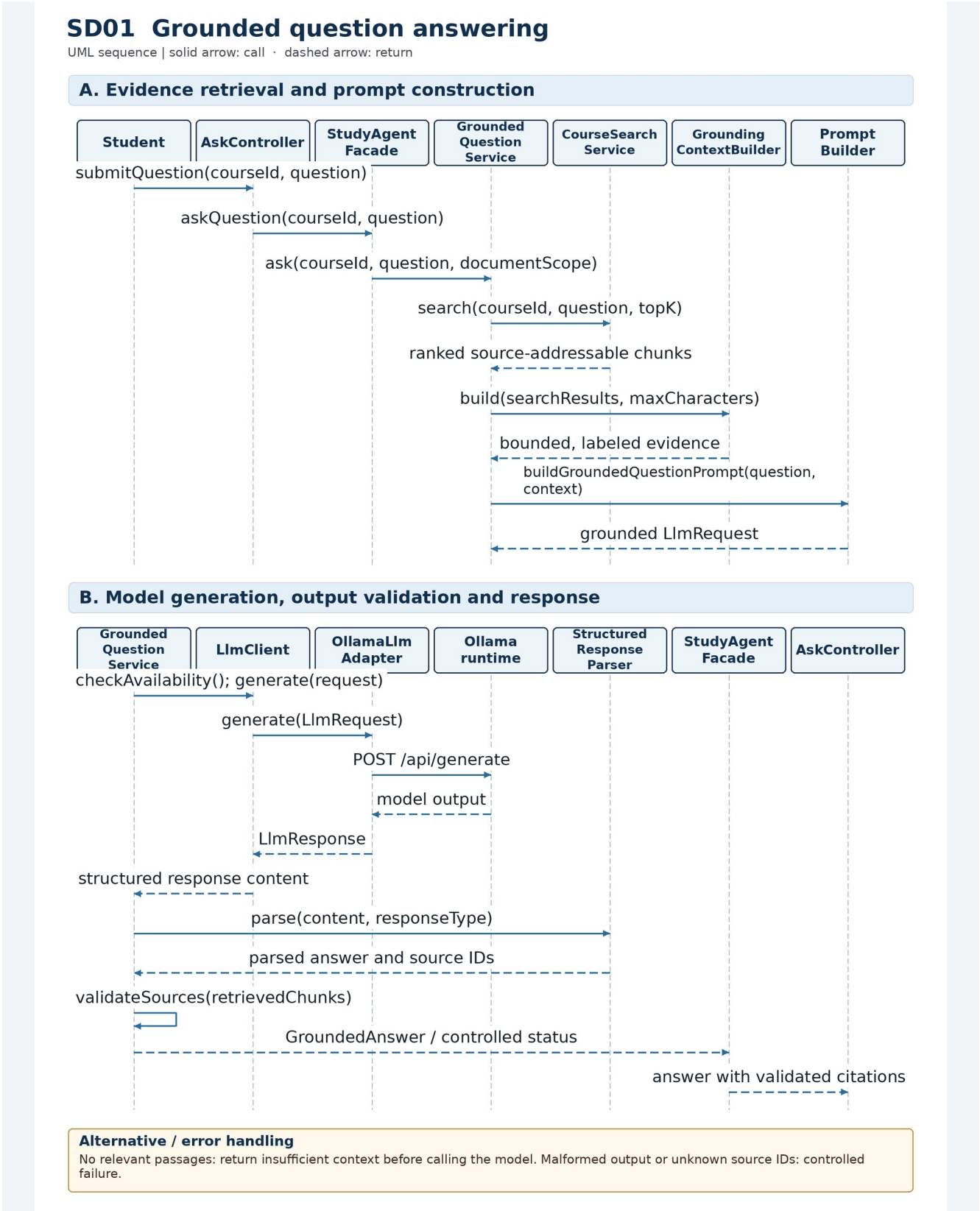


Stage 1 proposed use cases

5.3 Sequence diagrams

The sequence diagrams below show the interactions among the presentation, application, agent, AI integration and persistence components. The arrows carry the actual method calls, requests and returned values. Solid arrows represent calls; dashed arrows represent returns. Each diagram also identifies an important alternative or error case.

SD01 Grounded question answering

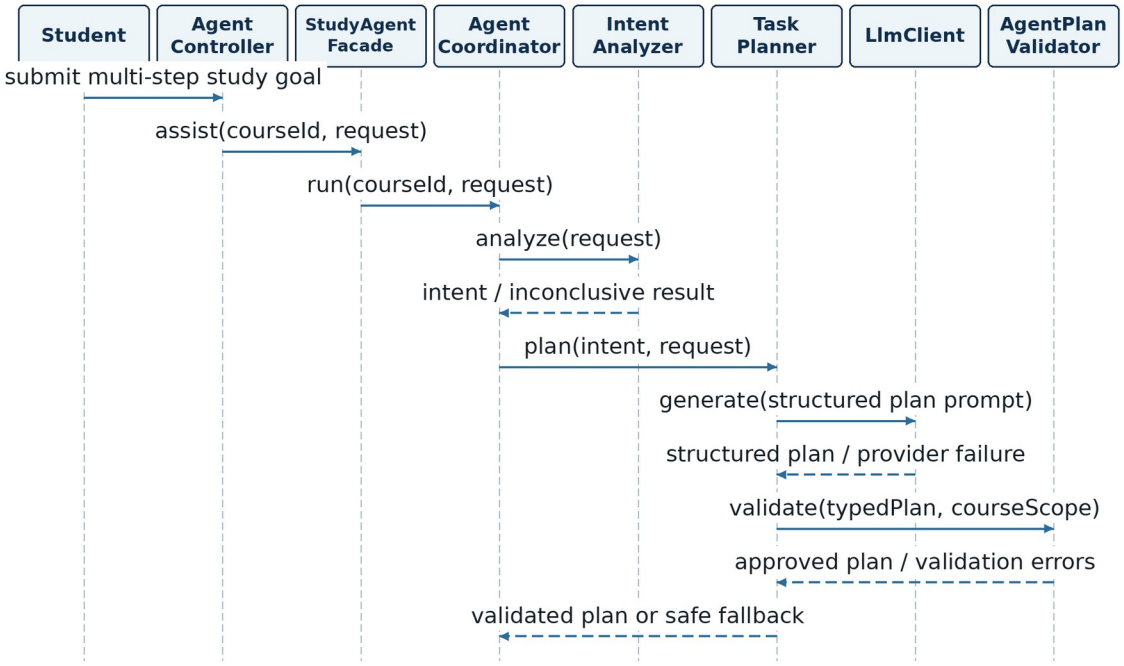


SD02 Bounded multi-step agent task

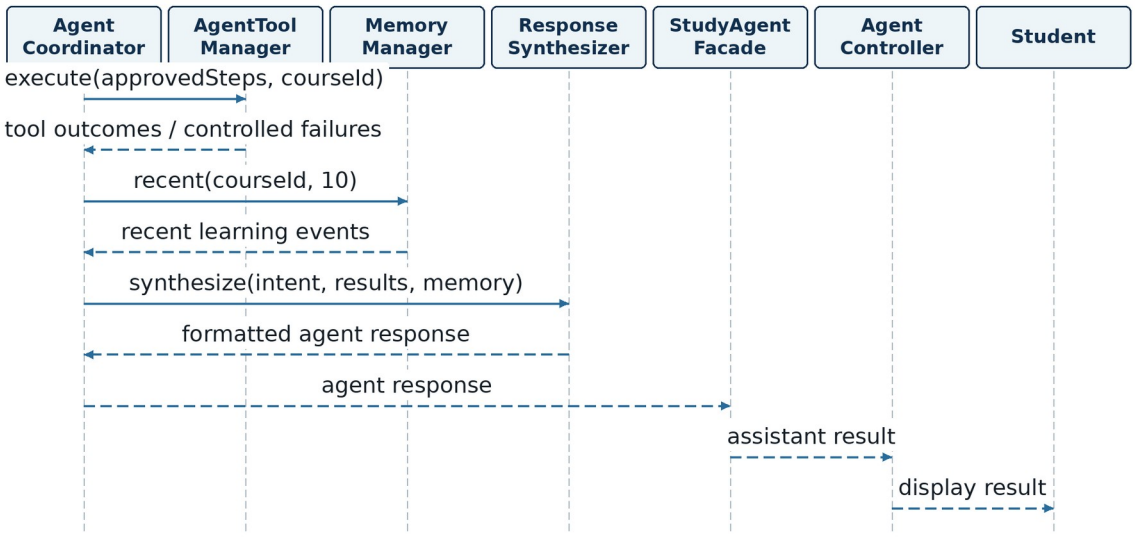
SD02 Bounded multi-step agent task

UML sequence | solid arrow: call · dashed arrow: return

A. Intent analysis and bounded plan proposal



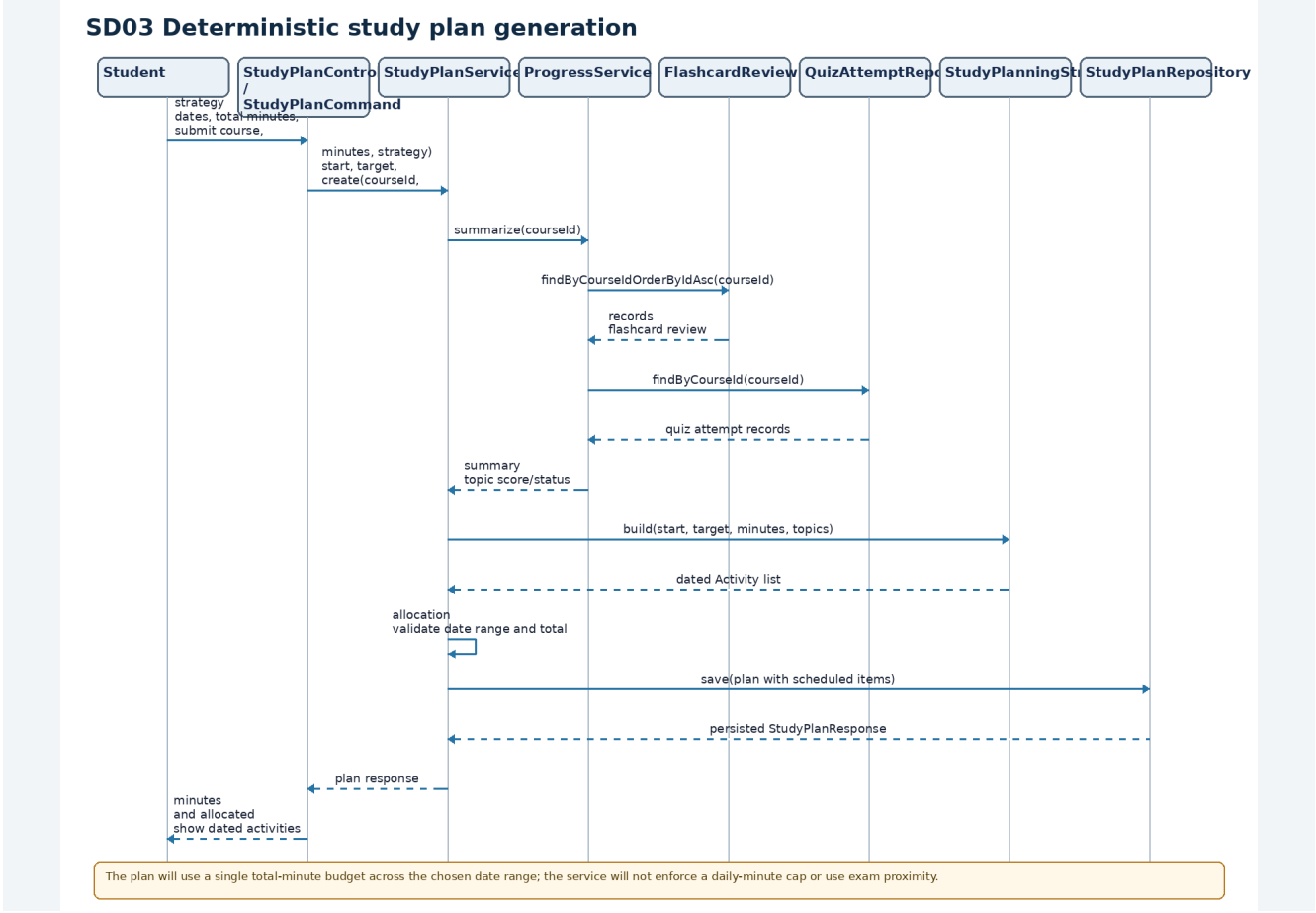
B. Tool execution, bounded memory and synthesis



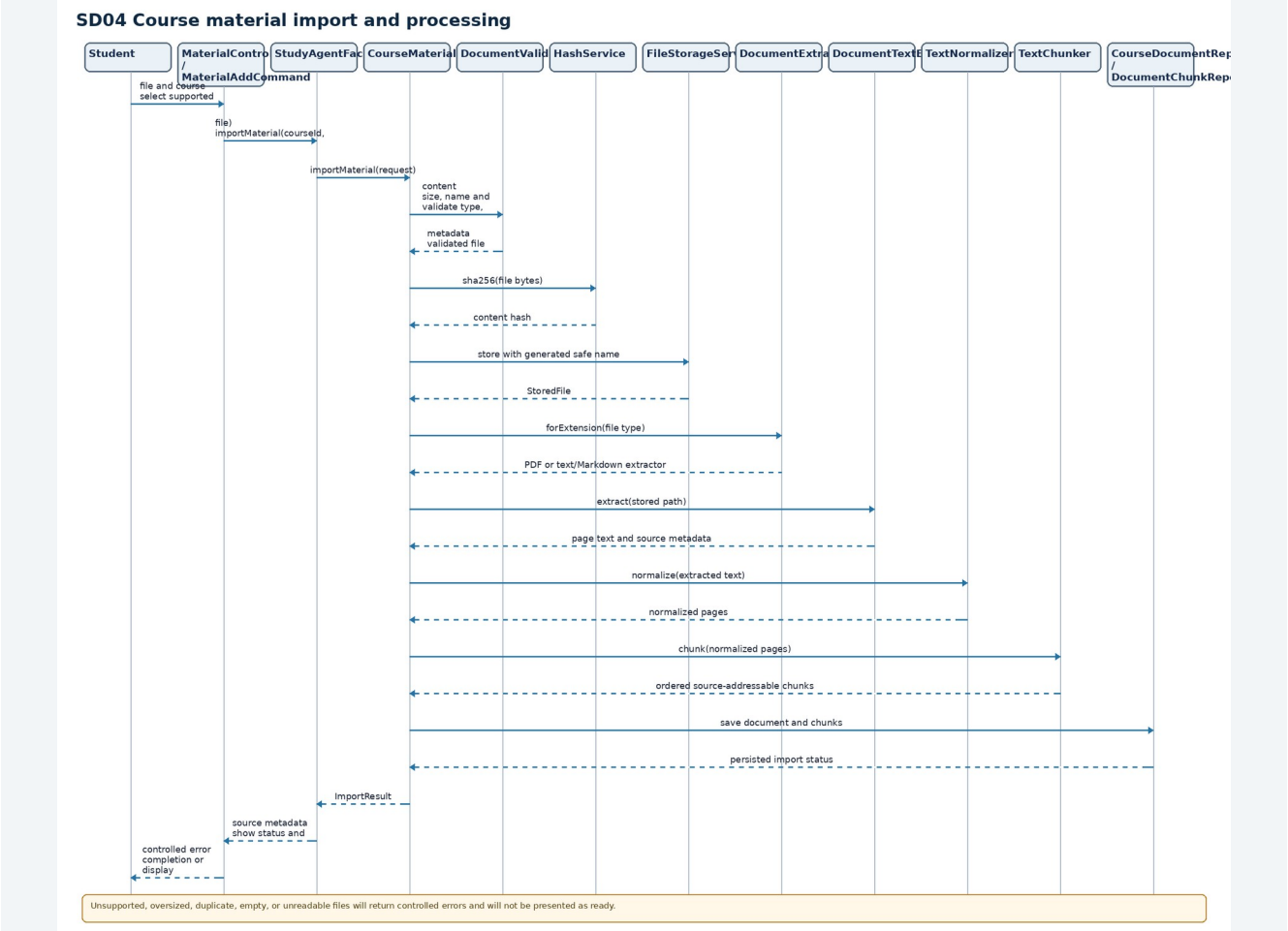
Alternative / error handling

If LLM planning fails, use a deterministic typed fallback. The validator checks course scope, allowlisted tools and the 1–5 step limit.

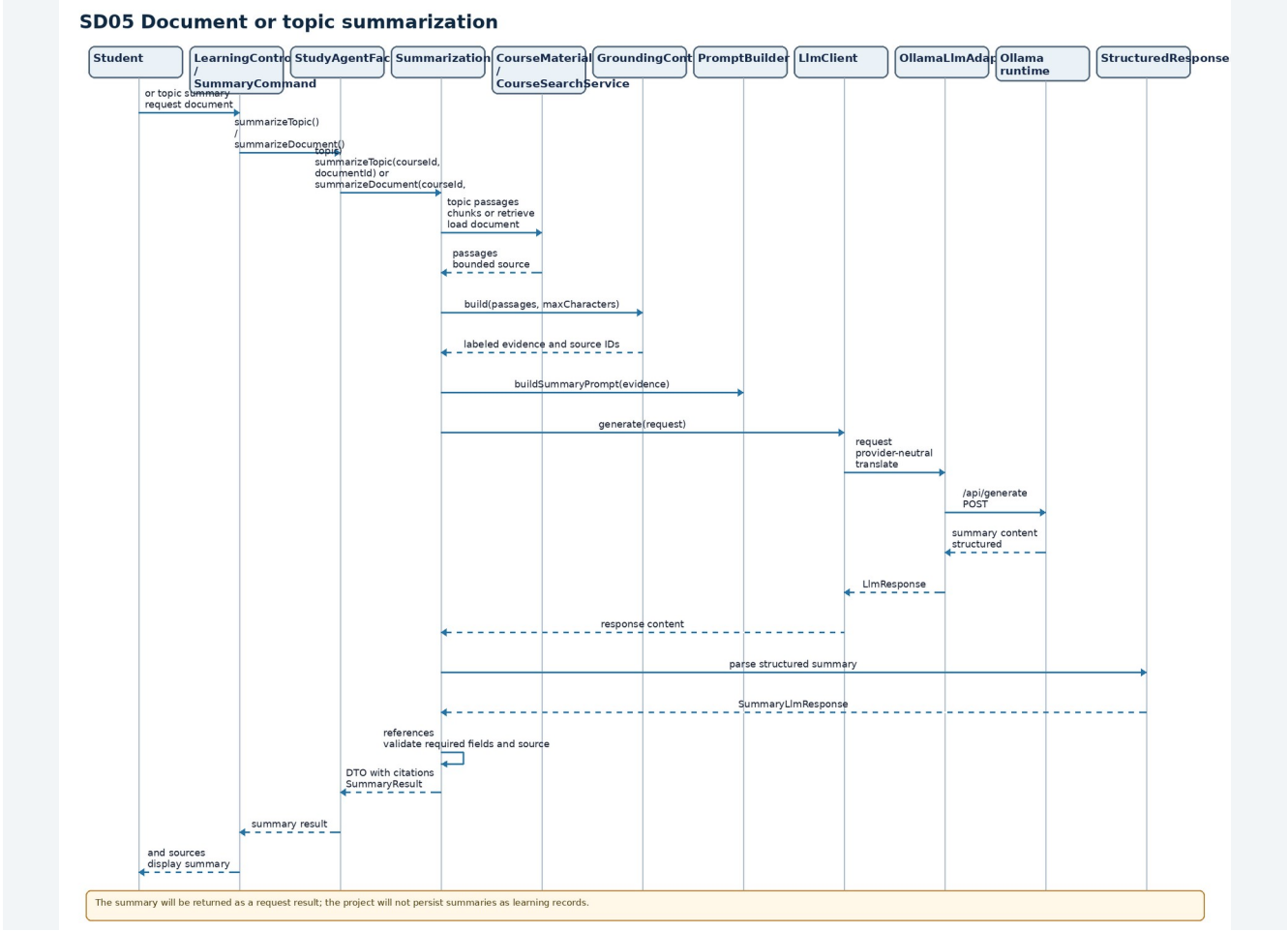
SD03 Study-plan generation



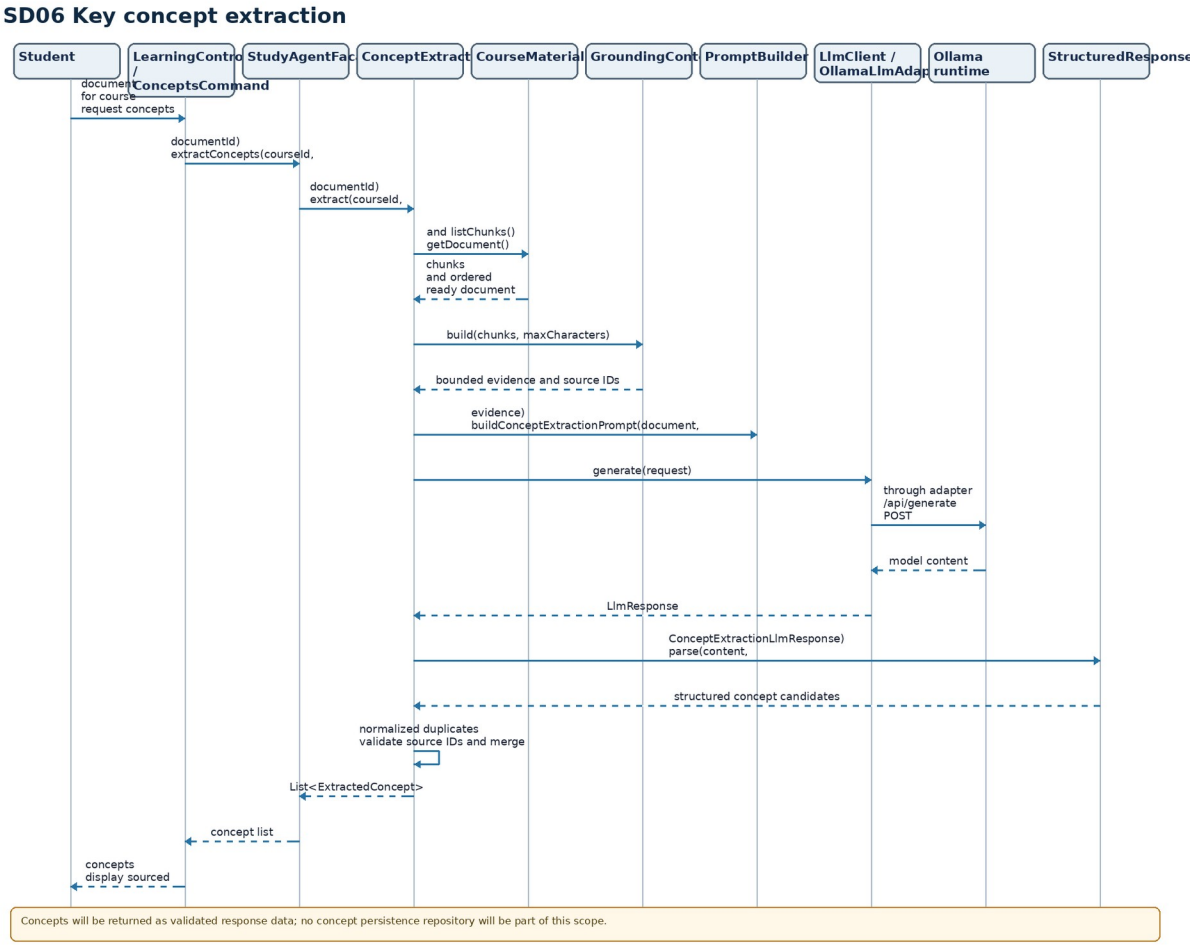
SD04 Import and retrieval pipeline



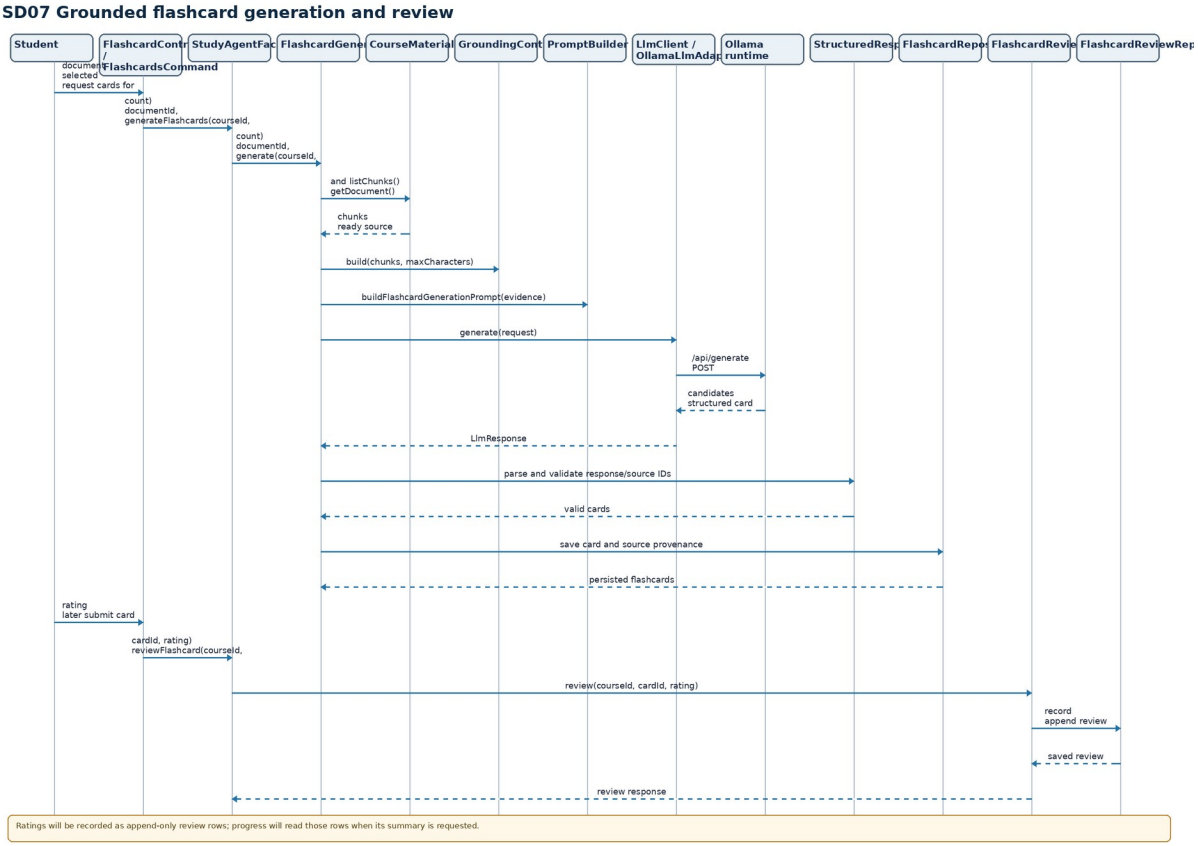
SD05 Grounded learning-content generation



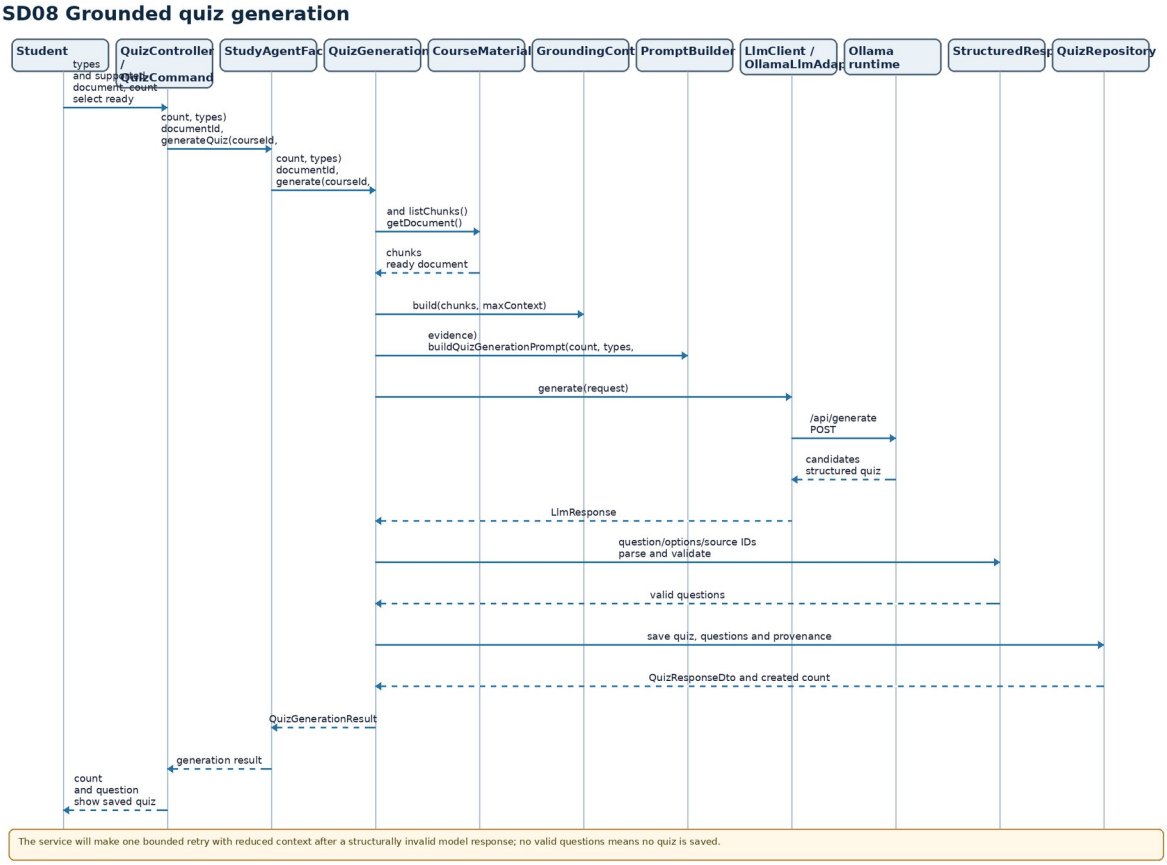
SD06 Key concept extraction



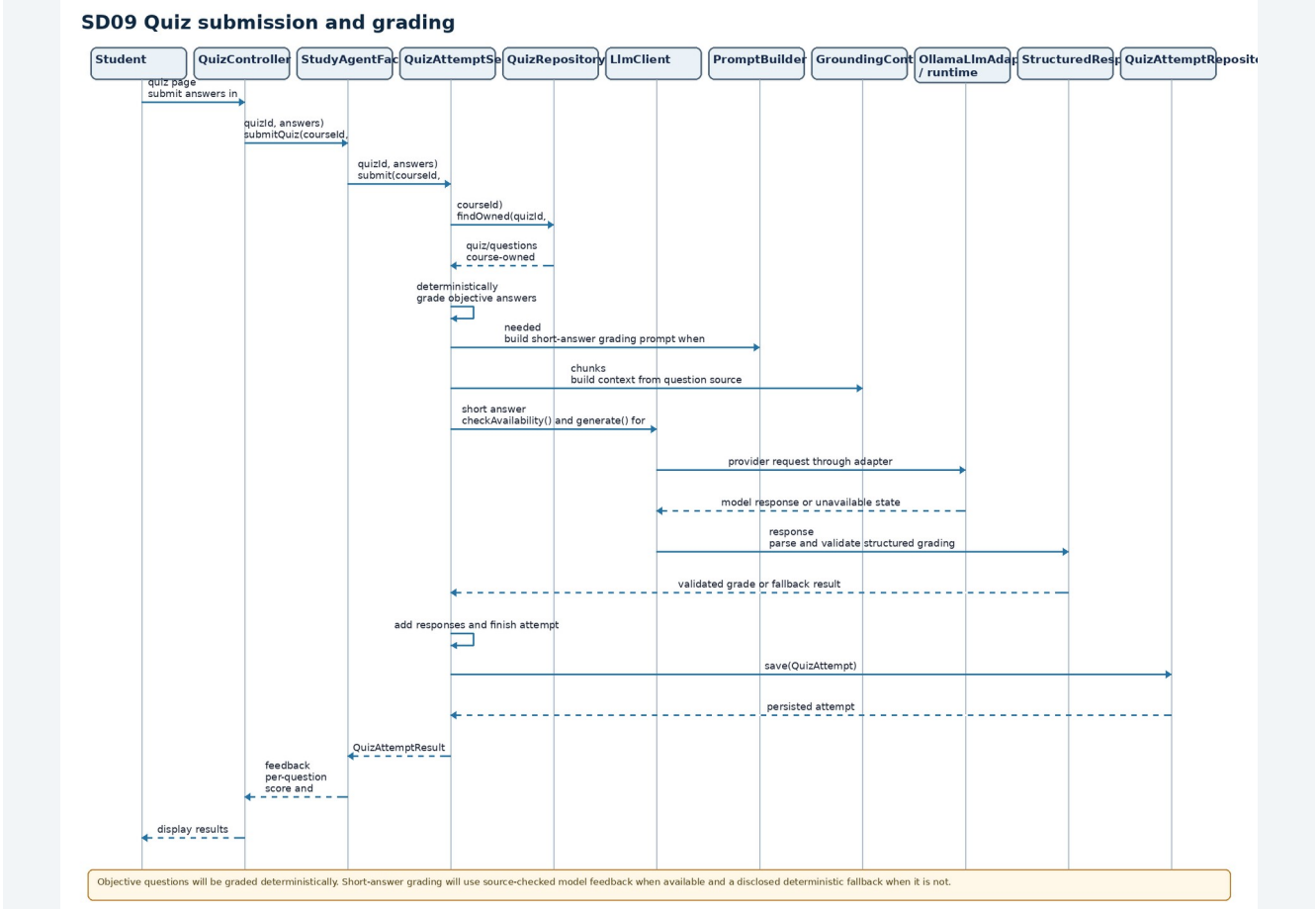
SD07 Grounded flashcard generation and review



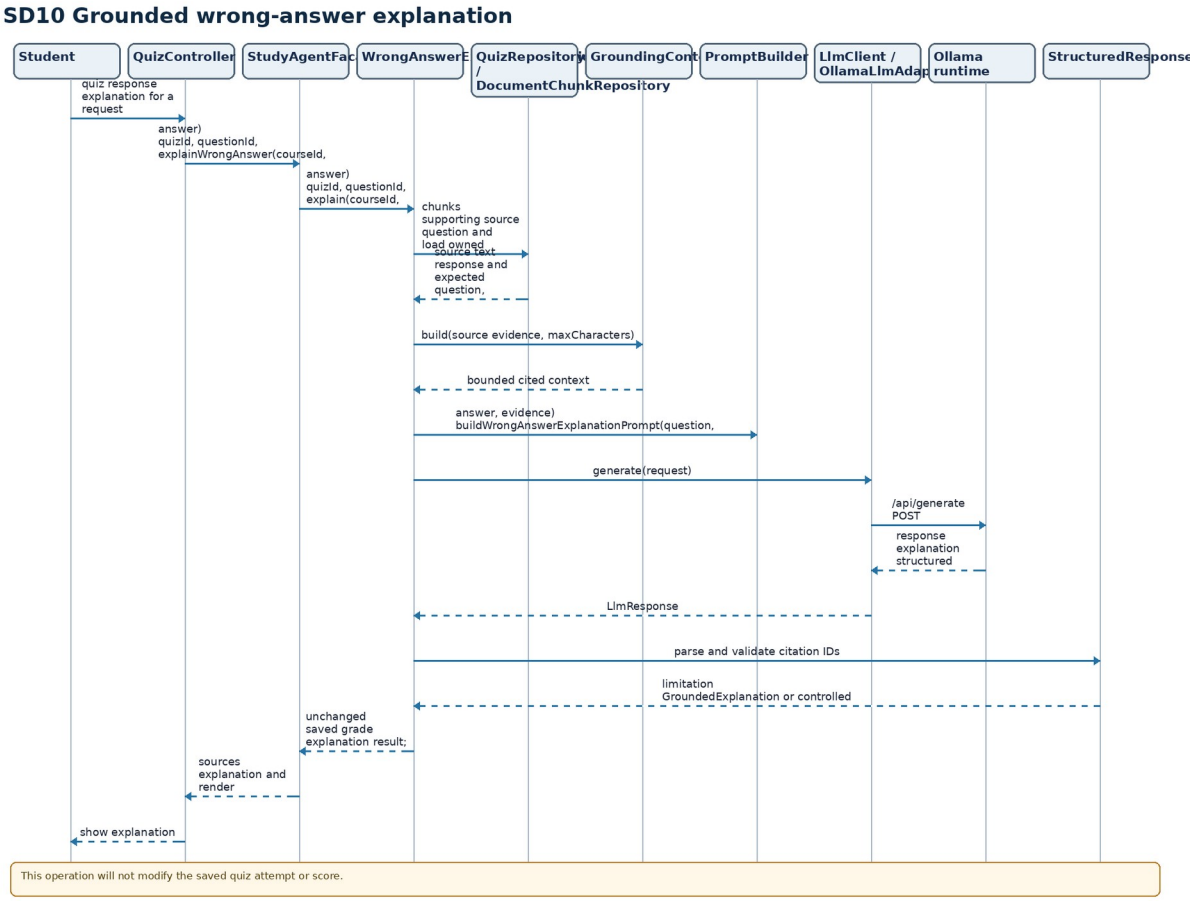
SD08 Grounded quiz generation



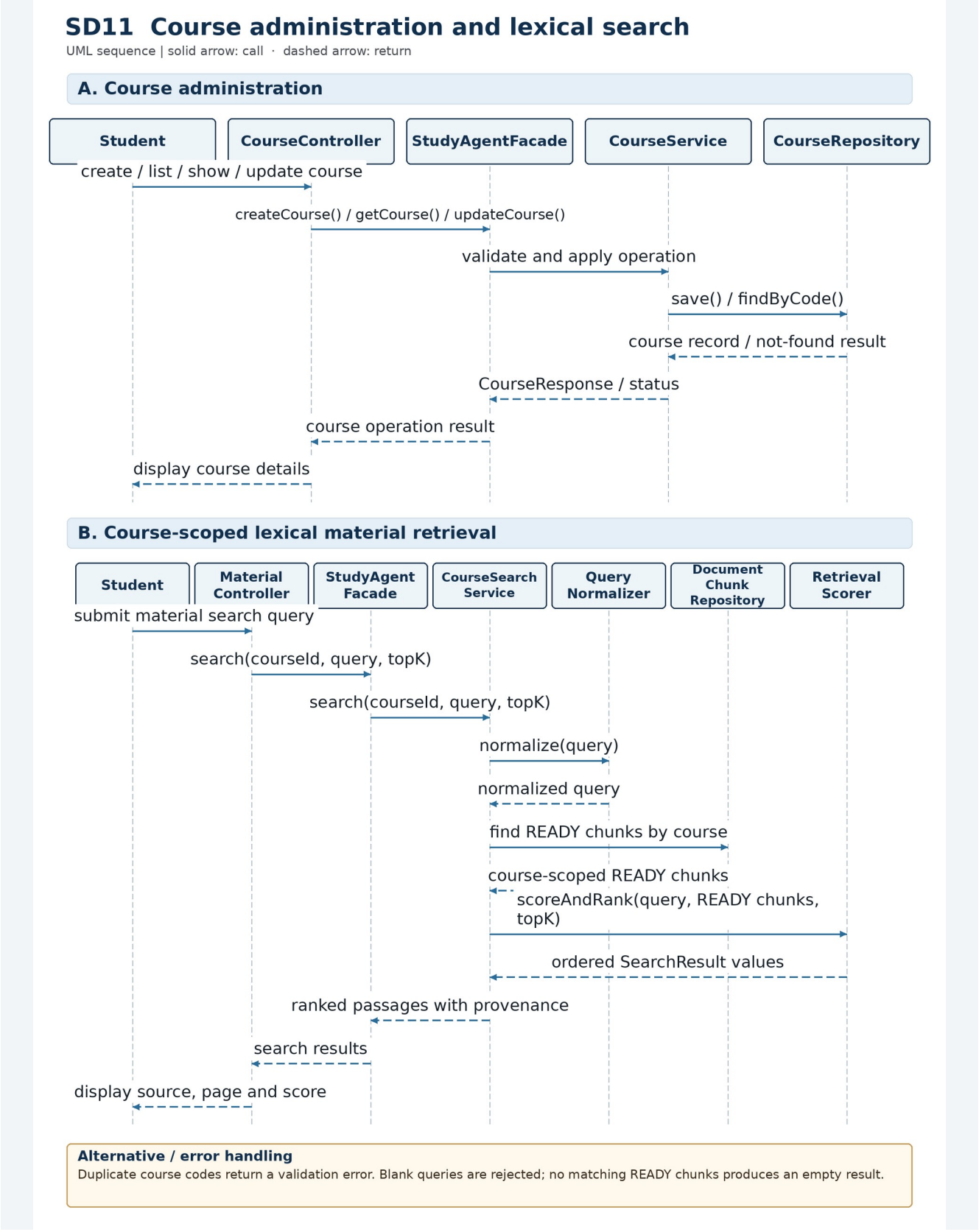
SD09 Quiz submission and grading



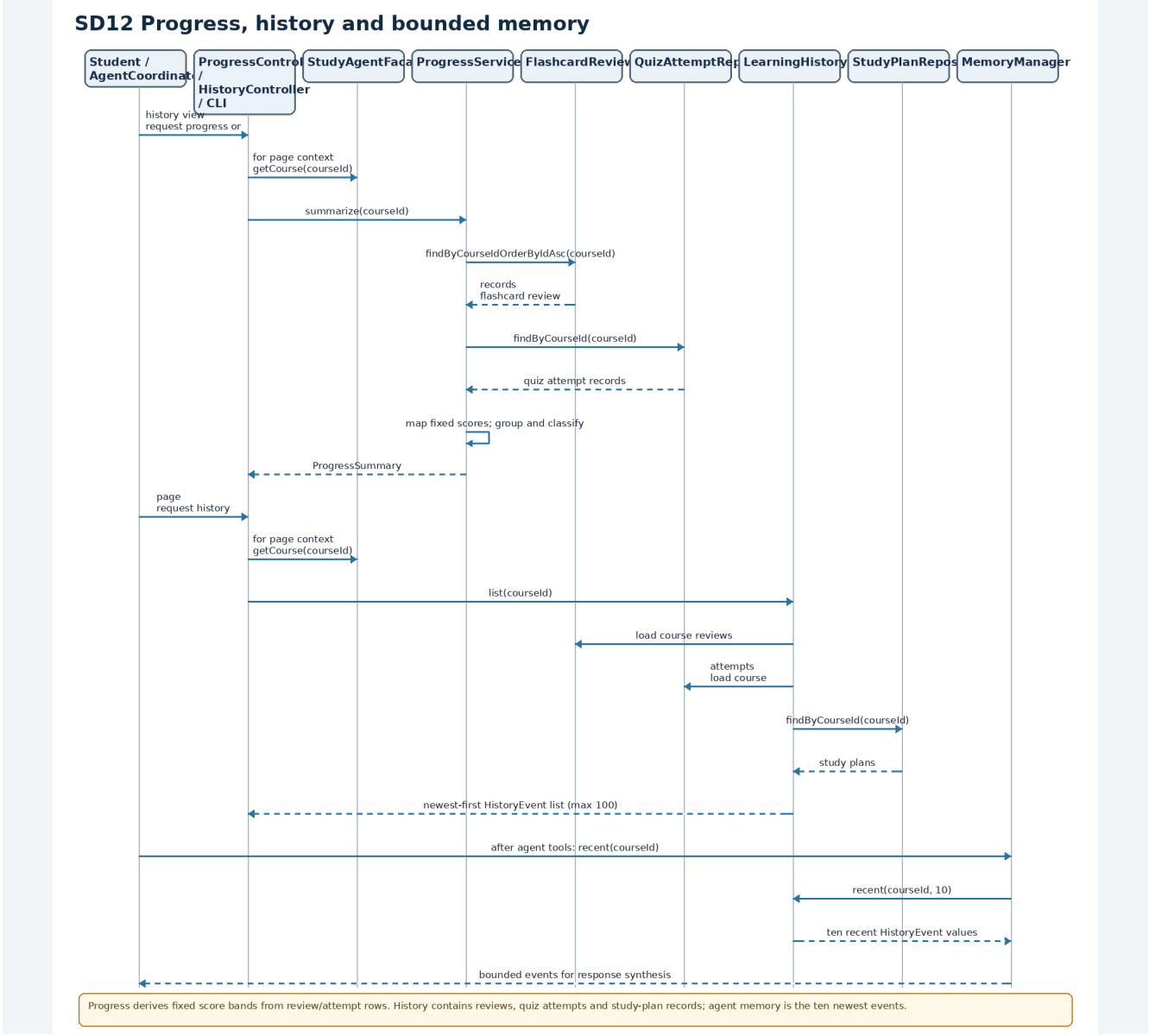
SD10 Grounded wrong-answer explanation



SD11 Course administration and lexical retrieval



SD12 Progress, history and bounded memory



6. Feature-to-design traceability matrix

Every proposed feature will map to at least one use case, named classes, callable operations, sequence flow and design pattern. Each sequence identifier will refer to the specific flow defined in Section 5.3.

Feature / classification	Use case · sequence · patterns	Participating classes / interfaces	Key operations
F01 — Course workspace management Create, view, update, and delete course records. Type: Deterministic	UC01 SD11 MVC · Facade · Command	CourseController; CourseService; CourseRepository; Course; CourseCreateCommand; CourseListCommand; CourseShowCommand	createCourse(), getAllCourses(), getCourse(), getCourseByCode(), updateCourse(), deleteCourse()
F02 — Course material import and processing Validate, extract, normalize, chunk, and store source files. Type: Deterministic	UC02 SD04 MVC · Facade	MaterialController; MaterialAddCommand; StudyAgentFacade; CourseMaterialService; DocumentValidator; HashService; FileStorageService; DocumentExtractorFactory; DocumentTextExtractor; TextNormalizer; TextChunker; CourseDocumentRepository; DocumentChunkRepository	importMaterial(), validate(), sha256(), store(), forExtension(), extract(), normalize(), chunk(), save()
F03 — Course material retrieval Search course-scoped indexed chunks with lexical retrieval. Type: Deterministic	UC03 SD11 MVC · Facade	MaterialController; MaterialSearchCommand; StudyAgentFacade; CourseSearchService; QueryNormalizer; RetrievalScorer; LexicalRetrievalScorer; DocumentChunkRepository	search(), searchDocument(), normalize(), score()
F04 — Grounded course question answering Answer course questions using retrieved source evidence. Type: Hybrid	UC04 SD01 MVC · Facade · Adapter	AskController; AskCommand; StudyAgentFacade; GroundedQuestionService; CourseSearchService; GroundingContextBuilder; PromptBuilder; LlmClient; OllamaLlmAdapter; StructuredResponseParser	askQuestion(), ask(), askDocument(), search(), build(), generate(), parse()
F05 — Document and topic summarization Summarize a selected document or topic with source grounding. Type: Hybrid	UC05 SD05 MVC · Facade · Adapter	LearningController; SummaryCommand; StudyAgentFacade; SummarizationService; CourseMaterialService; CourseSearchService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser	summarizeDocument(), summarizeTopic(), build(), generate(), parse(), validSources()

Feature / classification	Use case · sequence · patterns	Participating classes / interfaces	Key operations
F06 — Key concept extraction Extract and normalize key concepts from course material. Type: Hybrid	UC06 SD06 MVC · Facade · Adapter	LearningController; ConceptsCommand; StudyAgentFacade; ConceptExtractionService; CourseMaterialService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser; ExtractedConcept	extract(), getDocument(), listChunks(), build(), generate(), parse(), normalizeKey()
F07 — Adaptive flashcard generation and review Generate source-linked cards and record review outcomes. Type: Hybrid	UC07 SD07 MVC · Facade · Adapter	FlashcardController; FlashcardsCommand; StudyAgentFacade; FlashcardGenerationService; FlashcardReviewService; Flashcard; FlashcardSource; FlashcardReview; FlashcardRepository; FlashcardReviewRepository	generateFlashcards(), generate(), list(), reviewFlashcard(), review(), save()
F08 — Grounded quiz generation Generate and retain quiz questions grounded in course material. Type: Hybrid	UC08 SD08 MVC · Facade · Adapter	QuizController; QuizCommand.Generate; QuizCommand.ListQuizzes; StudyAgentFacade; QuizGenerationService; CourseMaterialService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser; QuizRepository	generateQuiz(), generate(), list(), build(), parse(), validate(), save()
F09 — Quiz taking and grading Record quiz responses and calculate deterministic scores. Type: Hybrid	UC09 SD09 MVC · Facade · Adapter	QuizController; StudyAgentFacade; QuizAttemptService; Quiz; QuizQuestion; QuizAttempt; QuizResponse; QuizRepository; QuizAttemptRepository; LlmClient; PromptBuilder; GroundingContextBuilder; StructuredResponseParser	submitQuiz(), submit(), grade(), deterministic(), addResponse(), finish(), save()
F10 — Evidence-based wrong-answer explanation Explain an incorrect answer using cited course evidence. Type: Hybrid	UC10 SD10 MVC · Facade · Adapter	QuizController; StudyAgentFacade; WrongAnswerExplanationService; QuizRepository; DocumentChunkRepository; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser	explainWrongAnswer(), explain(), findOwned(), findById(), build(), generate(), parse()
F11 — Progress and weak-area analysis Summarize recorded assessment and review activity. Type: Deterministic	UC11 SD12 MVC · Facade	ProgressController; ProgressCommand; StudyAgentFacade; ProgressService; ProgressSummary; FlashcardReviewRepository; QuizAttemptRepository	getProgress(), summarize(), status(), round()

Feature / classification	Use case · sequence · patterns	Participating classes / interfaces	Key operations
F12 — Personalized study-plan generation Create dated activities from progress and selected strategy. Type: Deterministic	UC12 SD03 MVC · Strategy	StudyPlanController; StudyPlanCommand.Create; StudyPlanCommand.ListPlans; StudyPlanService; StudyPlanningStrategy; BalancedStudyStrategy; WeakTopicFocusedStrategy; ExamCramStrategy; ProgressService; StudyPlanRepository	create(), list(), build(), summarize(), addItem(), save()
F13 — Adaptive next-step recommendations and agent task planning Plan bounded tasks and recommend a next study action. Type: Hybrid	UC13 SD02 MVC · Facade · Adapter · Command	AgentController; AgentCommand; RecommendationController; RecommendationCommand; StudyAgentFacade; AgentCoordinator; IntentAnalyzer; TaskPlanner; LlmClient; PromptBuilder; StructuredResponseParser; AgentPlanValidator; AgentToolManager; MemoryManager; ResponseSynthesizer; RecommendationService	assist(), recommend(), run(), analyze(), plan(), generate(), parse(), validate(), execute(), recent(), synthesize()
F14 — Learning history and bounded agent memory Present recent learning events and bounded agent context. Type: Deterministic	UC14 SD12 MVC · Facade	HistoryController; HistoryCommand; StudyAgentFacade; LearningHistoryService; MemoryManager; HistoryEvent; FlashcardReviewRepository; QuizAttemptRepository; StudyPlanRepository	listHistory(), list(), recent(), MemoryManager.recent()

7. Feature realization explanations

The following walkthroughs will connect the user action to the proposed classes and methods. Together with Section 6, they will provide a direct path from each requirement to the use case, runtime interactions and design structure.

F01 Course workspace management

Related use case: UC01 — Manage a course

Related sequence diagram: SD11

Proposed classes: CourseController; CourseService; CourseRepository; Course.

Key methods: createCourse(), updateCourse(), getCourseByCode(), deleteCourse()

CourseController will expose course create, list, show, update and delete actions. Picocli will expose create, list and show. Both paths will delegate supported actions through StudyAgentFacade to CourseService, which will normalize fields and enforce uniqueness before using CourseRepository. The GUI and CLI will return only the operations their interfaces expose.

F02 Course material import and processing

Related use case: UC02 — Import course material

Related sequence diagram: SD04

Proposed classes: MaterialController; CourseMaterialService; DocumentValidator; FileStorageService; DocumentExtractorFactory; DocumentTextExtractor; TextNormalizer; TextChunker; repositories.

Key methods: importMaterial(), validate(), sha256(), forExtension(), extract(), normalize(), chunk()

MaterialController or MaterialAddCommand will call StudyAgentFacade.importMaterial(). CourseMaterialService will coordinate DocumentValidator, hashing, FileStorageService, DocumentExtractorFactory, a PDF/TXT/MD extractor, TextNormalizer and TextChunker. It will persist document metadata and ordered chunks only after the processing pipeline passes validation.

F03 Course material retrieval

Related use case: UC03 — Search course material

Related sequence diagram: SD11

Proposed classes: CourseSearchService; QueryNormalizer; RetrievalScorer; LexicalRetrievalScorer; DocumentChunkRepository.

Key methods: search(), searchDocument(), normalize(), score()

MaterialSearchCommand or MaterialController will invoke CourseSearchService.search(). QueryNormalizer will create a normalized query; DocumentChunkRepository will provide course-scoped READY content; LexicalRetrievalScorer will score and sort matches. The result will preserve document and chunk provenance for display and downstream grounding.

F04 Grounded course question answering

Related use case: UC04 — Ask a grounded course question

Related sequence diagram: SD01

Proposed classes: AskController; StudyAgentFacade; GroundedQuestionService; CourseSearchService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser.

Key methods: askQuestion(), ask(), askDocument(), build(), generate(), parse(), validateSources()

AskController or AskCommand will call StudyAgentFacade.askQuestion(), which will delegate directly to GroundedQuestionService. CourseSearchService will retrieve evidence; GroundingContextBuilder and PromptBuilder will construct a bounded LlmRequest. LlmClient will route through OllamaLlmAdapter. StructuredResponseParser and source validation will reject unsupported citations before GroundedAnswer reaches the user. The Q&A workflow will not be described as an AgentTool.

F05 Document and topic summarization

Related use case: UC05 — Summarize a document or topic

Related sequence diagram: SD05

Proposed classes: SummarizationService; CourseMaterialService; CourseSearchService; GroundingContextBuilder; PromptBuilder; LlmClient; parser.

Key methods: summarizeDocument(), summarizeTopic(), build(), generate(), parse()

SummarizationService will load document chunks or retrieve topic results, build bounded evidence, and request a structured summary through LlmClient. The parser will validate summary fields and source IDs. The service will return a SummaryResult DTO; it will not persist summaries as learning records.

F06 Key concept extraction

Related use case: UC06 — Extract key concepts

Related sequence diagram: SD06

Proposed classes: LearningController; StudyAgentFacade; ConceptExtractionService; CourseMaterialService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser; ExtractedConcept.

Key methods: extract(), getDocument(), listChunks(), build(), generate(), parse(), normalizeKey()

ConceptExtractionService will load one READY document through CourseMaterialService, build bounded context, and request structured concepts through LlmClient. StructuredResponseParser will parse candidates; the service will validate source IDs, normalize names and merge duplicates. It will return ExtractedConcept DTOs without persisting them.

F07 Adaptive flashcard generation and review

Related use case: UC07 — Generate and review flashcards

Related sequence diagram: SD07

Proposed classes: FlashcardController; StudyAgentFacade; FlashcardGenerationService; FlashcardReviewService; Flashcard; FlashcardSource; FlashcardReview; FlashcardRepository; FlashcardReviewRepository; ProgressService.

Key methods: generateFlashcards(), generate(), list(), reviewFlashcard(), review()

FlashcardGenerationService will load the selected ready document, build bounded context, request structured cards, validate source IDs, remove duplicates and persist valid Flashcard and FlashcardSource records. FlashcardReviewService will list course-owned cards, validate card ownership and rating, then append a FlashcardReview. ProgressService will read that record on its next summary request; no Observer event will be claimed.

F08 Grounded quiz generation

Related use case: UC08 — Generate a quiz

Related sequence diagram: SD08

Proposed classes: QuizController; StudyAgentFacade; QuizGenerationService; CourseMaterialService; GroundingContextBuilder; PromptBuilder; LlmClient; StructuredResponseParser; QuizRepository; Quiz.

Key methods: generateQuiz(), build(), generate(), parse(), validateQuestion(), save()

QuizGenerationService will require a course-owned READY document, validate the requested count and supported question types, build bounded evidence, and request structured questions through LlmClient. It will validate answer structures and source IDs, deduplicate prompts, persist valid Quiz and QuizQuestion records, and disclose partial counts. The current form will not request a difficulty setting.

F09 Quiz taking and grading

Related use case: UC09 — Take and submit a quiz

Related sequence diagram: SD09

Proposed classes: QuizController; StudyAgentFacade; QuizAttemptService; Quiz; QuizQuestion; QuizAttempt; QuizResponse; QuizRepository; QuizAttemptRepository; LlmClient; PromptBuilder; GroundingContextBuilder; StructuredResponseParser.

Key methods: submitQuiz(), submit(), grade(), addResponse(), finish(), save()

QuizAttemptService.submit() will load a course-owned Quiz and create a QuizAttempt. Its grade() method will compare objective answers deterministically. For short answers it will build an evidence-bounded prompt, parse and validate returned source IDs, and otherwise use a deterministic overlap fallback with an explanatory status. QuizResponse records and the completed attempt will be saved. This workflow does not require an event publisher or listener.

F10 Evidence-based wrong-answer explanation

Related use case: UC10 — Request an answer explanation

Related sequence diagram: SD10

Proposed classes: WrongAnswerExplanationService; QuizRepository; DocumentChunkRepository; PromptBuilder; LlmClient; parser.

Key methods: explainWrongAnswer(), loadEvidence(), buildPrompt(), generate(), parse(), validateSources()

WrongAnswerExplanationService will confirm the quiz/question relationship and reload cited chunks through DocumentChunkRepository. PromptBuilder will combine the question, student answer, reference answer and evidence. The adapter-backed model output will be parsed and citation-checked. The explanation will be read-only with respect to the attempt score.

F11 Progress and weak-area analysis

Related use case: UC11 — Review progress and weak areas

Related sequence diagram: SD12

Proposed classes: ProgressController; StudyAgentFacade; ProgressService; ProgressSummary; FlashcardReviewRepository; QuizAttemptRepository.

Key methods: summarize(), status(), round(), hasActivity()

ProgressService.summarize() will read course flashcard reviews and quiz attempts. It will map ratings to fixed scores, calculate review-source averages and quiz performance, and label values below 60 as WEAK, 60–79.9 as DEVELOPING, and 80 or above as STRONG. No separate ProgressPolicy, trend calculation, or minimum sample-size rule will be specified.

F12 Personalized study-plan generation

Related use case: UC12 — Create a study plan

Related sequence diagram: SD03

Proposed classes: StudyPlanService; StudyPlanningStrategy; BalancedStudyStrategy; WeakTopicFocusedStrategy; ExamCramStrategy; ProgressService; StudyPlanRepository.

Key methods: create(), list(), build(), summarize(), addItem(), save()

StudyPlanService.create() will validate dates, positive total minutes and strategy name. It will load ProgressSummary topics and pass them to the selected strategy, which will create dated activities. The service will reject a sum above the one total-minute budget and persist the plan and items. It will not enforce daily caps or calculate exam proximity.

F13 Adaptive next-step recommendations and agent task planning

Related use case: UC13 — Ask agent for a recommendation or multi-step task

Related sequence diagram: SD02

Proposed classes: AgentController; AgentCommand; StudyAgentFacade; AgentCoordinator; IntentAnalyzer; TaskPlanner; LlmClient; PromptBuilder; StructuredResponseParser; AgentPlanValidator; AgentToolManager; MemoryManager; LearningHistoryService; ResponseSynthesizer; RecommendationService.

Key methods: assist(), recommend(), run(), analyze(), plan(), generate(), parse(), validate(), execute(), recent(), synthesize()

AgentCoordinator.run() will call IntentAnalyzer.analyze() first. Clear requests will map to a supported intent and its deterministic plan. For ambiguous or explicitly multi-step requests, TaskPlanner.plan() will build a bounded planning prompt and use LlmClient to request a structured plan proposal; StructuredResponseParser will convert valid output into typed steps. AgentPlanValidator will enforce course scope, the supported intent set, the tool allowlist and a one-to-five step limit before execution. Invalid planning output will fall back to the deterministic progress-review plan. AgentToolManager will execute services in plan order, MemoryManager will retrieve ten recent course events, and ResponseSynthesizer will format the validated outcomes.

F14 Learning history and bounded agent memory

Related use case: UC14 — Inspect learning history and memory

Related sequence diagram: SD12

Proposed classes: HistoryController; StudyAgentFacade; LearningHistoryService; MemoryManager; HistoryEvent; FlashcardReviewRepository; QuizAttemptRepository; StudyPlanRepository.

Key methods: listHistory(), list(courseId), recent(courseId, limit), MemoryManager.recent(courseId)

LearningHistoryService.list(courseId) will map flashcard reviews, quiz attempts, and study-plan creation records to HistoryEvent values, sort newest first, and cap the result at 100 entries. MemoryManager.recent(courseId) will request the ten most recent events. Neither topic filtering nor a separate MemoryContext object will be specified.